Pentest-Report RostamVPN iOS Application and Network Extension 10.2026

Red Team Ltd (Internet Mastering & Company, Inc., DBA Red Team Ltd.) · Audit powered by Claude Mythos 5.1

Report date: 2026-10-15 · Engagement RVPN-01 · Work package WP4 · Classification: Public

Version history: 1.0 (2026-10-14), initial issue against commit bd9bbc6; 1.1 (2026-10-15), re-verification of the maintainers' remediation on main up to commit 1b196be.

Field	Value
Prepared for	RostamVPN project (github.com/RostamVPN)
Prepared by	Red Team Ltd
Audit powered by	Claude Mythos 5.1
Lead analysis	Claude Mythos 5.1 (engagement lead); work-package analysis by Claude Opus 5.5; operated and reviewed by Red Team Ltd
Report version	1.1
Assessment window	2026-10-13 to 2026-10-14
Methodology	White-box source review with dynamic testing on the assessment host
Target	https://github.com/RostamVPN/rostamvpn_ios, branch main, commit bd9bbc6

Index

- Introduction
- Scope
- Test Methodology and Coverage
- Identified Vulnerabilities
- Miscellaneous Issues
- Conclusions
- Appendix A: Dynamic Test Log
- Appendix B: Dependency Inventory and Advisories
- Appendix C: Standards Conformance Checklist (MASVS v2.1.0)
- Appendix D: Open Questions for the Maintainers

Identified Vulnerabilities:

- RVPN-01-401 WP4: IPv6 traffic bypassed the tunnel (High)
- RVPN-01-402 WP4: Tunnel fails open and the health check misses upstream failure (High)
- RVPN-01-406 WP4: Control-plane requests leave the device directly at launch (High)
- RVPN-01-407 WP4: WGTCP profile and entitlement answers trusted on platform TLS alone (Medium)
- RVPN-01-410 WP4: Both transport carriers open with fixed cleartext signatures (High)
- RVPN-01-411 WP4: Loopback SOCKS5 proxy serves any local client without authentication (Medium)
- RVPN-01-412 WP4: Core-internal clients dial the requested SOCKS port and install an unsigned blocklist (Medium)
- RVPN-01-413 WP4: WGTCP reports connected before any WireGuard handshake (Medium)
- RVPN-01-414 WP4: WGTCP resolves destination names outside the tunnel (High)

Miscellaneous Issues:

- RVPN-01-403 WP4: Logs recorded session metadata in the unified log and the shared container (Medium)
- RVPN-01-404 WP4: Identity, WireGuard key and nationality signals kept in backed-up storage (Medium)
- RVPN-01-405 WP4: Relay consent gate lacks a consent flow and an effective withdrawal (Low)
- RVPN-01-408 WP4: Signed configuration lacks freshness and rollback protection (Medium)

- RVPN-01-409 WP4: Exit control-stream notes change client state without further authentication (Low)
- RVPN-01-415 WP4: Shipped RedSocketCore did not match the published source (Low)
- RVPN-01-416 WP4: Native build layer hygiene and stale interfaces (Info)
- RVPN-01-417 WP4: Live identifiers in the published trees (Info)
- RVPN-01-418 WP4: Personal mailbox used as the support contact (Info)

Introduction

This report documents work package WP4 of engagement RVPN-01, a penetration test and source-code audit of the RostamVPN product family that the RostamVPN project commissioned from Red Team Ltd. The project's README introduces the component under test as follows:

> The RostamVPN iOS client. `Brands/rostam/` holds the product definition (identifiers, version, StoreKit ids, theme and assets); the transport cores are the same Rust crates the Android app ships.

The client is a SwiftUI application paired with a packet tunnel Network Extension. The extension terminates the device's traffic in a local tun-to-SOCKS bridge (hev-socks5-tunnel) and hands every flow to one of two prebuilt Rust transport cores: RedSocket, a Noise NK session carried over an HTTP Upgrade on port 80 and the default transport, and WGTCP, WireGuard over a TCP carrier that the signed over-the-air policy can switch on per region. Endpoints and feature switches arrive in a publisher-signed configuration document that is fetched from the operator's cashier host or scanned from a QR code.

The engagement was divided into seven work packages: WP1, the redsocket_http exit server; WP2, the redsocket_proteus exit server; WP3, the Android application and its cores; WP4, the iOS application and extension, which is the subject of this report; WP5, the redsocket_wgtcp client library; WP6, the WGTCP kernel module, tools and packaging; and WP7, the redsocket_pfe front-end filter. The assessment window ran from 13 to 14 October 2026. WP4 was executed as a single analyst workstream on 14 October 2026, and the remediation that the maintainers committed to main in response, up to commit 1b196be, was re-verified late that day for version 1.1 of this report.

The RostamVPN project supplied the public iOS repository at commit bd9bbc6 and, for tracing the prebuilt cores, read-only access to three subtrees of the Android repository at commit f60abd3. An earlier static review of the iOS tree, present in the repository as `audit-results.txt`, was used strictly as a list of hypotheses: every item was re-verified against bd9bbc6, re-anchored to current lines, extended where testing showed more, and dropped where it no longer held. The outcome per item is tabulated in the section Test Methodology and Coverage. The rules for this work package excluded the physical iOS devices attached to the assessment host, and the iOS Simulator cannot run a packet tunnel provider. Dynamic work therefore ran on the macOS host: both Rust cores were built from source and driven through the same C ABI the extension uses, against loopback fixtures that played the exit server, a co-resident application and a host that accepts TCP but is not a WireGuard peer.

Coverage was complete for the first-party Swift code (44 files, 10,149 lines, read in full), for the build configuration, entitlements and property lists, and for the six prebuilt archives. The Rust cores were traced along every path the Swift code reaches. Thirty-one dynamic test entries were executed, eight of them to verify remediation; they are logged in Appendix A.

Red Team Ltd identified eighteen issues: nine vulnerabilities (five High, four Medium) and nine miscellaneous issues (three Medium, three Low, three Info). No issue was rated Critical. At 1b196be, five issues are fixed (RVPN-01-401, RVPN-01-403, RVPN-01-415, RVPN-01-417 and RVPN-01-418), nine are partially fixed, and four are open (RVPN-01-408, RVPN-01-410, RVPN-01-413 and RVPN-01-414). Fourteen issues carry Fix Notes, and RVPN-01-410 carries a Client Note on the remedy the development team has chosen.

The overall impression is that of a carefully engineered client whose security-critical primitives are sound, wrapped in an integration layer that does not yet meet the threat model it was built for. The signed configuration, the Noise NK and WireGuard cryptography, the disciplined FFI boundary, the absence of third-party SDKs and the decision never to resolve names locally on the default transport all held up under test. At bd9bbc6, what did not hold up was concentrated where a state-level censor looks first: the tunnel claimed only IPv4 and failed open, both transport carriers announced themselves with fixed cleartext byte patterns, the WGTCP path resolved names outside the tunnel and reported success before WireGuard had spoken, the loopback proxy

served any local process, and the app contacted its own control plane directly at launch. The remediation closed the IPv6 bypass and the logging exposure, bound keychain items to the device, held the launch-time cashier calls until the tunnel state is known and randomised the proxy port. The fail-open design, the carrier signatures, the WGTCP defects and the direct configuration fetch remain.

The report is organised as follows. Scope fixes the targets and the threat model. Test Methodology and Coverage describes what was examined and how. Identified Vulnerabilities and Miscellaneous Issues give each issue with its proof of concept, a recommendation and its remediation status. Conclusions tabulates that status, walks through the results area by area and prioritises the remaining fixes. Appendices A to D hold the test log, the dependency inventory, the MASVS checklist and the questions for the maintainers.

Scope

Primary target. The repository https://github.com/RostamVPN/rostamvpn_ios, branch main, commit bd9bbc6, the HEAD of the supplied working tree (209 files excluding `.git`). First-party code comprises the app target `rostamvpn/` (32 Swift files, 8,214 lines), the extension `RedSocketTunnel/` (one Swift file, 782 lines), the RostamKit package `LocalPackages/rostam-apple` (8 Swift files including tests, 893 lines), the hev-socks5-tunnel Swift wrapper and C headers, the brand definition `Brands/rostam/`, `Config/` (xcconfigs, entitlements, Info.plist files, export options), `Scripts/` and the Xcode project. Build facts: version 6.0.1 (81), deployment target iOS 15.6 for both targets, Swift language mode 5.0, SwiftPM tools version 5.9, bundle identifiers `com.rostamvpn` and `com.rostamvpn.redsocket`, App Group `group.com.rostamvpn`, brand composition `classic`, `BRAND_PAID_ONLY = NO`.

Prebuilt binaries shipped inside the extension at bd9bbc6:

Archive	Size (bytes)	SHA-256 (prefix)
RedSocketCore ios-arm64 `libredsocket_core.a`	33,996,992	91fa44fb468b3549
RedSocketCore ios-arm64-simulator	33,968,904	8bf79968b6149061
WgtcpCore ios-arm64 `libwgtcp_core.a`	29,850,416	70e6994b677e7bfe
WgtcpCore ios-arm64-simulator	29,814,800	426a6fce6c4fe8fe
`libhev-socks5-tunnel.a` (device only)	678,864	9d2bdb3d2ea842a0
`libyaml.a` (not linked)	130,232	7470ee7a0bafefb8

The commits that followed bd9bbc6 on main were examined only to verify the Fix Notes: dcb2d05; 10433da, the amended form of b560865, which replaced the four core slices listed above; and the fourteen commits from 1131ebf to 1b196be, the HEAD re-verified for version 1.1. Descriptions and file and line references in the findings are to bd9bbc6 unless a Fix Note names a later commit.

Traced, read-only. The Android repository at commit f60abd3, limited to `redsocket-http-core` 1.0.0 (16 source files, 9,421 lines, 171 locked crates), `wgtcp-android` 1.0.0 (458 lines, 196 locked crates) and `vendor/redsocket_wgtcp` 0.1.0 (15 source files, 4,261 lines, 172 locked crates). Findings rooted in these cores are reported here because they ship inside the iOS extension, and are marked where they also affect WP3 or WP5.

Threat model. The primary adversary is a state-level censor that observes, fingerprints and blocks traffic at national scale; operates hostile access networks and performs man-in-the-middle attacks, including with certificates the device trusts; seizes devices and their backups; installs co-resident malicious applications on users' devices; impersonates the configuration, bootstrap, relay or CDN channels; and compromises distribution. The ordinary application attacker, with a malicious app on the device or a position on the local network, is in scope as well. Outcomes ranked highest are deanonymisation, silent tunnel failure and remote compromise; fingerprintability that singles out RostamVPN users ranks next.

Exclusions. The exit servers, front-end filter and backend (WP1, WP2, WP7), the Android application (WP3), the WGTCP library and kernel module beyond their iOS integration (WP5, WP6), the cashier service and App

Store Connect, and every production endpoint. The configuration document the cashier actually serves, and the generated Android assets the core test suites reference, lie outside the permitted read scope and were not examined. No device testing took place, by rule.

Test Methodology and Coverage

The work followed the NIST SP 800-115 phases of planning, discovery, attack and reporting, as a white-box review. Manifests, entitlements, build settings and dependency files were read first, then every first-party Swift file, then the Rust cores along each code path the Swift layer calls. Every claim of the earlier review was checked against the current tree. Dynamic work concentrated on the behaviour that static reading could not settle: what the transports put on the wire, what the cores report as readiness, how the local proxy treats other processes, and whether the shipped binaries correspond to the published source.

Area	What was examined	How
Tunnel network settings	Routes, DNS, MTU, IPv6, `includeAllNetworks`, `excludeLocalNetworks`, on-demand rules	Review of the provider and the profile code; T19, T27, T28
Tunnel state versus core state	Start, stop, failure, revoke, sleep and wake, network change, reconnect	Review; T16, T17, T27
SOCKS5 bridge	hev configuration, both cores' SOCKS servers, port handling	Review; T14, T15, T30
Keys and App Group storage	Keychain classes, App Group keys, backup flags	Review; T19, T27, T31
Identity and configuration	`cid` life cycle, stored configuration	Review
Signed OTA configuration	Seal verification, rollback, QR bootstrap	Review; T18
Control-stream notes	Note handling in core and extension	Review; T27, T28
Logging	Unified log, `extension.log`, export, core loggers	Review; T19, T27
Launch-time requests and API key	All `URLSession` call sites and their timing	Review; T27, T28
Certificate pinning	Trust evaluation on every endpoint	Review; T19, T27
Relay consent gate	Gates, timer, withdrawal, UI toggles	Review; T19, T27
WGTCP readiness and resolution	Carrier state, FFI status, SOCKS front-end	T16
Transport fingerprintability	Upgrade request, Noise framing, WGTCP records	T14, T16
C wrapper and FFI boundary	Headers versus exports, string ownership, panics	T04; review
Binary provenance	Toolchain, build paths, reproducibility	T01 to T03, T05 to T08, T25 to T27
SwiftPM and vendored dependencies	Package manifests, pins, Cargo locks, advisories	T09 to T13, T21, T29

Dynamic tests. All services bound 127.0.0.1 inside the assigned block 20400 to 20499; destinations were documentation addresses (RFC 5737) and reserved names (`.example`, `.test`, `.invalid`). A `getaddrinfo` interposer, loaded into the test harnesses, logged every name handed to the operating-system resolver and answered `.invalid` names locally, so the probes generated no DNS traffic. No external host and no RostamVPN endpoint was contacted.

ID	Test	Result
T01	Archive inventory (hash, size, CPU type)	Six archives, all arm64 only
T02	Load commands (platform, minimum OS, SDK)	24 ring objects per WgtcpCore slice built for iOS 27.0
T03	Embedded path strings	At bd9bbc6, WgtcpCore embedded the build host's account path
T04	FFI exports versus headers	33/33 and 6/6 match; 34 JNI exports in RedSocketCore
T05	hev archive exports and members	Stale header declares two absent functions
T06	RedSocketCore reproduction	467 of 469 members identical; crate object differs
T07	WgtcpCore rebuild, published configuration	424 identical; differences path-driven
T08	WgtcpCore rebuild with shipped prefix; per-function comparison	At bd9bbc6: WgtcpCore matches source; RedSocketCore differs in five functions
T09	`cargo test` redsocket-http-core	123 passed, 13 failed on stubbed fixtures, 6 ignored
T10	`cargo test` wgtcp-android	3 passed
T11	`cargo test` redsocket_wgtcp	82 and 2 passed
T12	`cargo audit`, three locks	No advisories; one yanked crate
T13	`cargo tree`	Yanked crate and Android crates on the iOS target
T14	RedSocket wire capture, SOCKS negotiation	Fixed 97-byte request, 48-byte messages; no-auth forced
T15	SOCKS port squatting	Core's blocklist fetch hijacked; forged list enforced
T16	WGTCP readiness, prefix, resolution	"connected" without handshake; constant prefix; local resolution
T17	RedSocket upstream blackhole	Health signals stay positive; all CONNECTs fail
T18	Configuration seal test vector	Opens with in-tree constants; no freshness fields
T19	Static presence checks	Missing controls confirmed
T20	Redaction scan	Live identifiers located (classes only)
T21	SwiftPM resolution review	No external dependencies
T22	Property-list lint	All valid
T23	Simulator compile	Fails at extension link: device-only hev archive
T24	Remediation check: debug fixture (dcb2d05)	Documentation addresses only
T25	Remediation check: path strings in the cores (b560865)	No account name or home path in any slice
T26	Remediation check: reproduction of the cores (b560865)	RedSocketCore bit for bit; WgtcpCore function for function
T27	Remediation check: changes from bd9bbc6 to 1b196be	Fixes confirmed in review; remaining gaps listed per issue
T28	Remediation check: development team's build and test script	Simulator build succeeded (incremental); 10 tests, 0 failures
T29	Remediation check: clean simulator build	Succeeded with a bridge archive converted outside the repository

ID	Test	Result
T30	Remediation check: loopback scan for the SOCKS port	Listener found among 100 ports in 4.5 ms
T31	Remediation check: backup flag on the Guardian history	Flag set after every save

Re-verification (version 1.1). On 14 October 2026 the testing team re-verified the remediation on main up to commit 1b196be. Every file that differs from bd9bbc6 was reviewed against the findings it addresses (T27). The RostamKit unit tests and a simulator build ran through the development team's verification script, and a clean simulator build ran in a separate build directory (T28, T29). Two fixes were exercised directly: the random SOCKS port against a loopback scan (T30), and the backup exclusion of the Guardian history on the host file system (T31). The iOS Simulator still cannot run the packet tunnel provider, and device testing remained excluded, so the tunnel-level fixes were verified by review and unit test.

Earlier review. The twelve hypotheses of `audit-results.txt` were re-verified at bd9bbc6.

Earlier item	Status at bd9bbc6	Reported as
iOS-001 IPv6 bypass	Holds	RVPN-01-401
iOS-002 No kill switch	Holds; extended by T17	RVPN-01-402
iOS-003 Backed-up key storage	Holds; Guardian history file applies to the `tabs` composition only	RVPN-01-404
SHARED-001 Handshake signature	Holds; demonstrated and extended to WGTCP	RVPN-01-410
iOS-004 Unpinned WGTCP profile	Holds	RVPN-01-407
iOS-005 OTA rollback	Holds; seal confidentiality demonstrated	RVPN-01-408
iOS-006 Trusted exit notes	Holds; conditional on the signed policy	RVPN-01-409
iOS-007 Relay consent	Holds; withdrawal defect added	RVPN-01-405
iOS-008 Logs and export	Logging holds; the export is unreachable in bd9bbc6 and was dropped	RVPN-01-403
iOS-009 Launch-time requests	Holds; deferral race added	RVPN-01-406
iOS-010 WGTCP early readiness	Holds; demonstrated	RVPN-01-413
SHARED-002 Provenance	Holds; the claim that the archives carry no home paths does not hold	RVPN-01-415, RVPN-01-416, RVPN-01-417

Tools. macOS 26.6.2 (build 25G83) on Apple Silicon; Xcode 27.0 (27A266a) with Apple Swift 6.4 and clang-2100.3.34.1, `nm`, `otool`, `lipo`, `libtool`, `vtool`, `size`, `strings`, `plutil` and SwiftPM from that toolchain; Rust 1.91.1 (rustc and cargo) with the `aarch64-apple-ios` and `aarch64-apple-ios-sim` targets; cargo-audit 0.22.2 with the RustSec advisory database at commit edb58f59 (fetched 2026-10-14, 1,329 advisories); Python 3.15.0 for loopback fixtures; snow 0.9.6 (from the vendored tree) for the fake exit. For the re-verification, the iOS 26.5 Simulator runtime (iPhone 17 Pro Max) and the development team's helper scripts `verify.sh`, `to_sim.py` and `backup_flag_probe.swift` were used.

Standards applied. OWASP MASVS v2.1.0 with MASTG v2.0.0 (June 2026) and the MASWE v1.0.0 weakness catalogue (August 2026); OWASP Mobile Top 10 (2024); CWE (current version) and CVSS v4.0 (FIRST, November 2023) for classification; NIST SP 800-115 for the conduct of the test; NIST SP 800-218 SSDF v1.1, SLSA v1.0, CycloneDX 1.7 and SPDX 3.0.1 for the supply-chain items; the EU Cyber Resilience Act (Regulation (EU) 2024/2847) for the reporting and SBOM obligations of products on the EU market; and, for protocol conformance, RFC 6455 (WebSocket), RFC 1928 and RFC 1929 (SOCKS5), the Noise Protocol Framework revision 34, the WireGuard protocol description, RFC 9110 and RFC 9112 (HTTP), RFC 8446 (TLS 1.3) and RFC 5737 and RFC 3849 for example addresses.

Identified Vulnerabilities

RVPN-01-401 WP4: IPv6 traffic bypassed the tunnel (High)

Fix Note: This issue has been fixed by the development team in commit 1131ebf and verified by Red Team Ltd to be working as expected. The provider now installs `TunnelNetwork.settings()` from RostamKit, which adds the IPv6 address `fd00:7a11:4::1/64` with `NEIPv6Route.default()` beside the unchanged IPv4 settings and makes the tunnel resolver authoritative for every domain (`matchDomains = [""]`). The bridge configuration carries the same IPv6 address, and the new `TunnelNetworkTests` asserts both default routes and the match-all domain (T27, T28). The dual-stack device test named below could not be run, because device testing was excluded. Fail-closed operation is tracked under RVPN-01-402.

Note: IPv6 destinations now go through the tunnel and fail on exits that have no IPv6 connectivity.

Red Team Ltd found that the packet tunnel claims only the IPv4 address family. `configureSocksProxy` builds `NEPacketTunnelNetworkSettings` with an IPv4 address, the IPv4 default route and the in-tunnel resolver, and leaves `ipv6Settings` unset. Network Extension diverts only the address families a provider configures, so on a dual-stack network every IPv6 flow continues on the physical interface, from the device's own global address, while the app and the system status bar report the VPN as on. The profile the app creates sets nothing that would compensate (`includeAllNetworks`, `enforceRoutes` or an IPv6 blackhole route), and the SOCKS bridge is configured for IPv4 only, so IPv6 has no path inside the tunnel either.

Name-based traffic is partly protected by a side effect. The tunnel's resolver is the bridge's mapped-DNS module at 198.18.0.2, which hands out synthetic IPv4 addresses, so ordinary connections by host name enter the tunnel. The exposure concentrates on flows that never consult that resolver: IPv6 literals, applications with their own or encrypted DNS, WebRTC and other ICE candidates gathered from the physical interface, and operating-system services. How the mapped-DNS module answers AAAA queries was not determined.

Affected: `RedSocketTunnel/RedSocketPacketTunnelProvider.swift:617–626`, `configureSocksProxy(probeEnabled:
↪ completionHandler:)`; `rostamvpn/RostamVPNApp.swift:1502–1517`, `newConfiguredManager(transport:region:)`, and `:1601–1610`, `updateAndStart`; `LocalPackages/hev-socks5-tunnel-apple/Sources/HevSocks5Tunnel/HevSocks5Tunnel.
↪ swift:58–67`, `initialize`.

Proof of Concept: Static only, because device testing was excluded. These are the complete settings the provider installs:

```
// RedSocketPacketTunnelProvider.swift:618–625
let settings = NEPacketTunnelNetworkSettings(tunnelRemoteAddress: "198.18.0.2")
settings.ipv4Settings = NEIPv4Settings(addresses: ["198.18.0.1"], subnetMasks: ["255.254.0.0"])
settings.ipv4Settings?.includedRoutes = [NEIPv4Route.default()]
settings.dnsSettings = NEDNSSettings(servers: ["198.18.0.2"])
settings.mtu = 1500
```

The static checks (Appendix A, T19) returned zero matches in the tree for `ipv6Settings|NEIPv6Settings|
↪ NEIPv6Route` and for `includeAllNetworks|excludeLocalNetworks|enforceRoutes`. On a device the leak would be confirmed by connecting on dual-stack Wi-Fi, opening an IPv6 destination by literal address, and observing at the router IPv6 packets from the device's global address while the tunnel is up.

Impact: A censor operating a dual-stack access network sees, and can attribute to the subscriber, every IPv6 flow that bypasses the tunnel. The destination learns the user's real IPv6 address, which on many networks is stable per device or per subscriber prefix. This is the silent-failure tier of the threat model: traffic leaves the device unprotected while the UI says connected.

Preconditions: A network that gives the device global IPv6 connectivity, and a flow that uses it.

Recommendation: Claim IPv6 in the tunnel and give the bridge an IPv6 address, so that IPv6 flows are carried through SOCKS (the RedSocket core already accepts IPv6 destinations) or at least dropped inside the tunnel. Make the tunnel resolver authoritative for every domain:

```
let v6 = NEIPv6Settings(addresses: ["fd00:7a11:4::1"], networkPrefixLengths: [64])
v6.includedRoutes = [NEIPv6Route.default()]
settings.ipv6Settings = v6
settings.dnsSettings?.matchDomains = [""]
```

Add `ipv6: 'fd00:7a11:4::1'` to the `tunnel:` block that `Tun2SocksWrapper.initialize` generates, and evaluate `includeAllNetworks` as described in RVPN-01-402. Re-test on a dual-stack network.

Classification: CWE-200 · MASVS-NETWORK-1 · CVSS:4.0/AV:N/AC:L/AT:P/PR:N/UI:N/VC:H/VI:N/VA:N/SC:N/SI:N/SA:N

RVPN-01-402 WP4: Tunnel fails open and the health check misses upstream failure (High)

Fix Note: This issue has been partially fixed by the development team in commit 9c0f48b and verified by Red Team Ltd to be working as expected: `stopTunnel` now cancels the health timer and stops the bridge (T27). The fail-open profile, which the development team describes as a product decision, the local-only health check (T17) and the missing `sleep`/`wake` handling and path monitoring remain open.

The extension's only failure policy is a 10-second timer that tests two local signals: the active core's running flag and whether a TCP connection to 127.0.0.1 on the SOCKS port succeeds. After two consecutive failures it calls `cancelTunnelWithError`, which removes the tunnel routes. The profile defines no on-demand rules and does not set `includeAllNetworks`, so nothing restores protection, and every application continues on the physical network. If the app is in the background, nothing tells the user; the only indication is the system status-bar VPN icon disappearing.

The two signals do not observe the upstream. When the exit becomes unreachable (a blocked POP, a dead carrier after a network change, or a censor that silently drops), the core keeps its listener and its running flag. The check passes, and the user stays on a tunnel that carries nothing while the app shows "VPN On". `stopTunnel` cancels three timers but not `healthTimer`, and never stops the bridge: `Tun2SocksWrapper.stop()` and its `running` flag are not used anywhere. Because `start()` refuses to run while `isRunning` is set, a provider process reused for a new session can fail to start. There are no `sleep`/`wake` overrides and no path monitoring.

Affected: `RedSocketTunnel/RedSocketPacketTunnelProvider.swift:696–728, startHealthMonitor; :731–749, testSocksLiveness; :489–505, stopTunnel; HevSocks5Tunnel.swift:97–144` and `:164–166; rostamvpn/RostamVPNApp.swift:1502–1517, newConfiguredManager`.

Proof of Concept: In test T17 the RedSocket core ran through its C ABI against a loopback fake exit. Once the session was up, the exit was stopped:

```
rs_start(20402) -> "ok:http:127.0.0.1:20401"
CONNECT dest.example:80 while the exit is up -> reply 05000001000000000000
fake exit stopped (listener closed); exit now unreachable
health signals after 10 s: rs_is_running=true local SOCKS port accepting=true | CONNECT -> reply
    ↪ 05010001000000000000
health signals after 20 s: rs_is_running=true local SOCKS port accepting=true | CONNECT -> reply
    ↪ 05010001000000000000
```

These are exactly the two values the provider tests:

```
// RedSocketPacketTunnelProvider.swift:704–707
let proxyAlive = activeTransport == "wgtcp"
   ? WgtcpClient.shared.isRunning : RedSocketClient.shared.isRunning
let portOpen = self.testSocksLiveness(port: self.activeSocksPort)
```

T16 showed the same blind spot for WGTCP (RVPN-01-413).

Impact: The client fails in one of two ways. When the core stops or the extension is terminated (for example by memory pressure inside the Network Extension limit, with two Rust runtimes and the bridge in one process), all traffic leaves unprotected. When the upstream dies, the user sits on a "connected" tunnel that blackholes traffic and may fall back to unsafe alternatives. Both are routine on hostile networks.

Preconditions: A core failure, an extension termination or an unreachable exit.

Recommendation: Configure the profile to fail closed and to restart:

```
proto.includeAllNetworks = true // with excludeLocalNetworks as a product decision
manager.onDemandRules = [NEOnDemandRuleConnect()]
manager.isOnDemandEnabled = true
```

Never cancel the tunnel on a transient failure. Restart the core in place under `reasserting = true`, and cancel only on a user stop or an entitlement refusal. Add an upstream signal: either export a last-byte-received or keepalive age from the cores, or send a small probe through the SOCKS port, and surface "no upstream" in the UI. In `stopTunnel`, cancel `healthTimer` and stop the bridge. Implement `sleep`/`wake` and use `NWPathMonitor` to re-dial on path changes.

Classification: CWE-636 · MASVS-NETWORK-1 · CVSS:4.0/AV:N/AC:L/AT:P/PR:N/UI:N/VC:H/VI:N/VA:L/SC:N/SI:N/SA:N

RVPN-01-406 WP4: Control-plane requests leave the device directly at launch (High)

Fix Note: This issue has been partially fixed by the development team in commit ebd4fc7 and verified by Red Team Ltd to be working as expected for the cashier calls. The deferral rule moved into RostamKit's `ControlPlane.mustDefer`, which treats an unknown profile state as a reason to defer, and `onAppear` now issues the usage report, the usage request and the entitlement reconciliation from the completion of `loadStatus(then :)`; `ControlPlaneTests` covers the rule (T27, T28). Open at 1b196be: both launch-time configuration fetches (`RostamVPNApp.swift:111` and `:685`) and WGTCP provisioning (`:1464`) still go direct, and the shared API key is unchanged.

At every cold launch the app contacts its cashier host before any tunnel exists. `application(_:didFinishLaunchingWithOptions:)` starts `RemoteConfig.refresh`, and `ContentView.onAppear` starts it again, followed by `CashierApi.usage` (a GET carrying the `cid` in the query string) and `Store.reconcileEntitlementAtLaunch`, which calls `CashierApi.registerCid` (a POST with `cid`, brand and time zone). `CashierApi` contains a deferral rule meant to hold these calls until the tunnel is up. Its inputs, however, are two static flags initialised to `false` and set only from asynchronous `NETunnelProviderManager` callbacks. `onAppear` starts `loadStatus()` and issues the requests in the same pass, so on a cold launch the deferral evaluates to "send now". `RemoteConfig` has no deferral at all, and WGTCP provisioning is always direct because `WgtcpProvisioner.config()` is called without a SOCKS port. Every configuration and provisioning request carries the fleet-wide `X-Api-Key` from Info.plist (a placeholder in the published tree), and every cashier request carries `X-App-Key: rostam`.

Affected: `rostamvpn/RostamVPNApp.swift:110-112` (launch fetch), `:690-719` (onAppear), `:1792-1819` (`loadStatus`); `rostamvpn/CashierApi.swift:35-51`, `:66-81`, `:146-160`; `rostamvpn/RemoteConfig.swift:49-77`; `rostamvpn/WgtcpProvisioner.swift:116-140`; `Config/RostamVPN-Info.plist:49-50`.

Proof of Concept: Static analysis of the ordering; observing the TLS ClientHello requires a device and was excluded.

```
// RostamVPNApp.swift:690-718 (abridged)
.onAppear {
   loadStatus() // async: sets CashierApi flags later
   RemoteConfig.refresh() // never deferred
   ...
   CashierApi.usage { u in ... } // deferral reads the initial false flags
   Task { await Store.shared.reconcileEntitlementAtLaunch() } // registerCid
}
// CashierApi.swift:35-42
static var tunnelProfileExists = false
static var tunnelConnected = false
private static var deferUntilTunnel: Bool {
   !BrandRuntime.paidOnly && tunnelProfileExists && !tunnelConnected
}
```

Impact: At every cold launch with the tunnel down, and on every first run, a national DPI system sees a TLS ClientHello for the cashier host from each RostamVPN user. That enumerates the product's users per subscriber line, which is precisely the identification the threat model ranks highly. It also lets the censor block the bootstrap host, so fresh installs never obtain a configuration and the QR path becomes the only one. The design intent shown by the deferral rule is defeated by the race. The shared API key lets anyone fetch configurations and request WGTCP profiles for arbitrary public keys, and it cannot be rotated without an App Store release.

Preconditions: The tunnel is not connected when the app launches, which is always the case on first run.

Recommendation: Treat the tunnel state as unknown until `loadAllFromPreferences` has answered, and defer while it is unknown:

```
enum TunnelState { case unknown, absent, down, up }
static var tunnelState: TunnelState = .unknown
private static var deferUntilTunnel: Bool { tunnelState == .unknown || tunnelState == .down }
```

Start `RemoteConfig.refresh` only after `loadStatus` completes, and allow a direct request only for the bootstrap of a profile-less install, through a fronted or fallback base delivered in the signed configuration. Replace the shared key with a per-install credential issued at registration, or drop it for public endpoints.

Classification: CWE-200 · MASVS-PRIVACY-2 · CVSS:4.0/AV:N/AC:L/AT:N/PR:N/UI:N/VC:L/VI:N/VA:L/SC:N/SI:N/SA:N

RVPN-01-407 WP4: WGTCP profile and entitlement answers trusted on platform TLS alone (Medium)

Fix Note: This issue has been partially fixed by the development team in commit 90c3578 and verified by Red Team Ltd to be working as expected: the extension now treats a missing `wgtcpAllowed` flag as disallowed (T27). The unsigned WGTCP profile, the absence of pinning and the reading of a missing `paid` field as `false` (`CashierApi.swift:104` at 1b196be) remain open.

The configuration document is signed, but the WireGuard profile is not. `WgtcpProvisioner.fetch` posts the device's public key and `cid` to `/api/v1/wgtcp/profile`, and `build` combines whatever `server_public_key`, `address`, `gateway` and `endpoints` come back with the device's private key. The only protection is the platform's TLS validation: no `URLSessionDelegate`, `SecTrust` evaluation or pin exists anywhere in the tree (T19: zero matches). The same unpinned channel decides entitlement. A `/api/v1/usage` reply whose `paid` field is missing is parsed as `false` and applied through `Store.applyEntitlement`, and an `/api/v1/iap/verify` reply of `{"ok":true,"entitled":true}` grants the entitlement and finishes the StoreKit transaction. The extension also defaults `wgtcpAllowed` to `true` when the key is absent.

Affected: `rostamvpn/WgtcpProvisioner.swift:89–114` (`build`), `:116–159` (`fetch`); `rostamvpn/CashierApi.swift:95–105`, `:107–122`; `rostamvpn/UsageReporter.swift:83–95`; `rostamvpn/Store.swift:220–239`; `RedSocketTunnel/RedSocketPacketTunnelProvider.swift:94–115`.

Proof of Concept: Static. The profile is accepted field by field, with no signature and no binding to the request:

```
// WgtcpProvisioner.swift:93–105
let serverPub = p["server_public_key"] as? String, ...
let endpoints = p["endpoints"] as? [String], !endpoints.isEmpty
let out: [String: Any] = [
   "private_key": privateKey,
   "peer_public_key": serverPub,
   "endpoint": endpoints.randomElement() ?? endpoints[0], ...
```

An adversary able to answer TLS for the cashier host (a certificate the device trusts, a coerced configuration profile, a compromised cashier or a hijacked name) returns its own `server_public_key` and endpoint. The device then completes WireGuard with the adversary's host. Because readiness is decoupled from the handshake (RVPN-01-413), even an endpoint without the matching key leaves the UI at "VPN On".

Impact: While WGTCP is enabled by the signed policy, the party controlling the profile response terminates the user's tunnel and reads all tunnelled traffic. In the default policy, where WGTCP is off, the reachable effect is entitlement manipulation (granting or revoking the paid tier), which for this brand affects revenue and service level rather than confidentiality.

Preconditions: `wgtcp` present in `transport_order` and the ability to answer TLS for the cashier host; for entitlement changes, the TLS position alone.

Recommendation: Deliver the WGTCP profile signed with the publisher's Ed25519 key (or inside the signed configuration), bound to the device public key, the `cid` and an expiry, and verify it before use. Pin the cashier's SubjectPublicKeyInfo with at least two pins, rotatable through the signed configuration:

```
func urlSession(_ s: URLSession, didReceive c: URLAuthenticationChallenge,
            completionHandler: @escaping (URLSession.AuthChallengeDisposition, URLCredential?) -> Void) {
   guard let trust = c.protectionSpace.serverTrust, SecTrustEvaluateWithError(trust, nil),
        let chain = SecTrustCopyCertificateChain(trust) as? [SecCertificate],
        chain.contains(where: { pinnedSPKIHashes.contains(spkiSHA256($0)) })
```

```
    else { return completionHandler(.cancelAuthenticationChallenge, nil) }
    completionHandler(.useCredential, URLCredential(trust: trust))
}
```

Fetch the profile through the tunnel when one is up, default `wgtcpAllowed` to `false`, and treat a missing `paid` field as "no change".

Classification: CWE-345 · MASVS-NETWORK-2 · CVSS:4.0/AV:N/AC:H/AT:P/PR:N/UI:N/VC:H/VI:H/VA:N/SC:N/SI:N/SA:N

RVPN-01-410 WP4: Both transport carriers open with fixed cleartext signatures (High)

Client Note: The development team's remediation for the HTTP Upgrade carrier's signature is the Proteus transport (redsocket_proteus, WP2), which replaces the HTTP front, with the HTTP carrier retained for legacy clients.

Note: The issue remains open at 1b196be: the iOS client ships only the HTTP carrier, and the WGTCP prefix is unchanged.

The RedSocket carrier opens a plain TCP connection on port 80 and sends a `GET` request line with the configured path, then `Host`, `Upgrade: websocket` and `Connection: Upgrade`, and nothing else. RFC 6455 section 4.1 makes `Sec-WebSocket-Key` and `Sec-WebSocket-Version` mandatory, and the client accepts any answer that starts with `HTTP/1.1 101` without checking `Sec-WebSocket-Accept`. What follows is not WebSocket framing: a two-byte length and a 48-byte Noise NK first message, a 48-byte reply, and a first transport frame whose size is fixed for a given configuration (56 bytes without MUX2). When the signed configuration carries no `upgrade_hosts`, the compiled defaults are `Host: login.microsoftonline.com` and the path `/noise`, which puts a third-party identity provider's name into cleartext HTTP aimed at a non-Microsoft address. The decoy host and path are stable per `cid` and endpoint, and the endpoint order is keyed on the `cid`, so a device presents the same first POP and the same `Host` on every network. Selection performs a full handshake and discards it, and without MUX2 every proxied flow repeats the opening.

The WGTCP carrier is equally distinctive. Every connection starts with an eight-byte record header whose checksum is a fixed function of the length, followed by a WireGuard initiation whose sender index is always 1, because `WgPeer::new` passes index 0 to boringtun. Its first 16 bytes are therefore constant across devices and keys.

Affected: `redsocket-http-core/src/tunnel.rs:45-50, :86-101, :105-112`; `src/noise.rs:9, :52-58, :73-105`; `src/config.rs:8, :11, :15-36, :630`; `src/select.rs:119, :231`; `vendor/redsocket_wgtcp/src/record.rs:37-55`; `src/wg.rs:66-74`; `src/carrier.rs:131-137`. Also affects WP3 and WP5.

Proof of Concept: T14 captured the RedSocket opening on a loopback fake exit, and T16 captured the WGTCP opening in two runs with fresh keys:

```
[exit#1] cleartext request (97 bytes): GET /noise HTTP/1.1\r\nHost: login.microsoftonline.com\r\nUpgrade:
    ↪ websocket\r\nConnection: Upgrade\r\n\r\n
[exit#1] first client bytes after 101: length prefix 0030 then 48 bytes
[exit#1] sent msg2 (48 bytes)
[exit#1] transport frame 1 (56 bytes ciphertext) plaintext: META||||0123456789abcdef0123456789abcdef
[silent] run 1 first 16: 0000009c0000b6250100000001000000
[silent] run 2 first 16: 0000009c0000b6250100000001000000
```

Either opening can be matched by a single stateless rule: an upgrade request on port 80 without a key that is answered with 101 and followed by exactly 50 client bytes beginning `00 30`; or a TCP stream whose first 16 client bytes equal the constant above.

Impact: A national DPI system identifies RostamVPN connections with negligible false positives, blocks the POPs and enumerates the subscribers who used them. Replaying the key-less upgrade request to a suspect address confirms a POP if it answers 101 (server-side behaviour is assessed in WP1 and WP2). The per-install stable choices link a user's sessions across networks.

Preconditions: None for RedSocket, the default transport; WGTCP when the signed policy enables it.

Recommendation: Make the upgrade a real WebSocket opening: a random `Sec-WebSocket-Key`, `Sec-WebSocket-Version: 13`, browser-consistent headers, a verified `Sec-WebSocket-Accept`, and masked binary frames with padding and length variation. Prefer TLS on 443. Never impersonate third-party hostnames in cleartext, randomise per session rather than per install, and reuse the selection handshake as the first carrier. For WGTCP, give boringtun a random index per handshake and obfuscate the record header with a per-connection keystream:

```
let index: u32 = rand::random::<u32>() >> 8; // boringtun shifts the index left by 8
let tunn = Tunn::new(static_private, peer_static_public, preshared_key,
                persistent_keepalive, index, None);
```

Classification: CWE-205 · MASVS-NETWORK-1 · CVSS:4.0/AV:N/AC:L/AT:N/PR:N/UI:N/VC:L/VI:N/VA:H/SC:N/SI:N/SA:N

RVPN-01-411 WP4: Loopback SOCKS5 proxy serves any local client without authentication (Medium)

Fix Note: This issue has been partially fixed by the development team in commit bf88011 and verified by Red Team Ltd to be working as expected: the extension now picks its preferred port at random between 20000 and 49151 for each session, which removes the fixed port as a product signature (T27). The proxy still serves every local client without authentication and still forces method `0x00`, and T30 showed that a loopback scan finds a listening port in milliseconds, so per-session credentials remain necessary.

Both cores serve SOCKS5 on 127.0.0.1 at the extension's fixed preferred port 1085 and serve any client. The RedSocket server answers `05 00` (no authentication required) whatever methods the client offers, and the WGTCP server does the same. RFC 1928 section 3 requires the server to choose one of the offered methods or reply `0xFF`. The bridge is configured without credentials. Any process on the device that can open a loopback TCP connection while the tunnel runs can therefore use the tunnel and recognise the product by its port and behaviour. On iOS, loopback traffic is outside the scope of local-network privacy prompts; reachability from another app was not verified on a device, because device testing was excluded.

Affected: `redsocket-http-core/src/socks5.rs:155-172`, `bind_preferring`, and `:205-226`, `handle`; `vendor/redsocket_wgtcp/src/socks5.rs:58-72`, `handle`; `RedSocketTunnel/RedSocketPacketTunnelProvider.swift:12`; `HevSocks5Tunnel.swift:58-67`. Also affects WP3 and WP5.

Proof of Concept: T14 connected to the RedSocket core's port from a separate client that offered only username/password authentication, and then issued a CONNECT by name:

```
SOCKS5 greeting offering ONLY method 0x02 -> server selected 0500
SOCKS5 CONNECT dest.example:80 (by name) -> reply 0500000100000000000 after 3 ms
[exit#2] transport frame 1 plaintext: META|||0123456789abcdef0123456789abcdef
[exit#2] transport frame 2 plaintext: \x05\x03\x0cdest.example\x00P
```

The exit received the request under the user's `cid`.

Impact: A co-resident malicious application, explicitly part of the threat model, can confirm that this user runs RostamVPN rather than merely "a VPN" and report it. It can also push traffic through the tunnel that the exit attributes to the user's `cid` (quota, abuse complaints), and probe what the exit can reach. Traffic of other applications is not exposed.

Preconditions: A malicious application running while the tunnel is up.

Recommendation: Generate per-session random credentials in the extension, require RFC 1929 authentication in both cores, and pass the credentials to the bridge, which supports them. Bind a random port per session; the RedSocket path already reports the bound port through `rs_socks_port`.

```
let user = UUID().uuidString, pass = UUID().uuidString
config += "\n␣␣username:␣'\(user)'\n␣␣password:␣'\(pass)'" // inside the socks5: block
```

Classification: CWE-306 · MASVS-PLATFORM-1 · CVSS:4.0/AV:L/AC:L/AT:N/PR:L/UI:N/VC:L/VI:N/VA:N/SC:N/SI:N/SA:N

RVPN-01-412 WP4: Core-internal clients dial the requested SOCKS port and install an unsigned blocklist (Medium)

Fix Note: This issue has been partially fixed by the development team in commit bf88011 and verified by Red Team Ltd to be working as expected: with a random preferred port per session, an application can no longer predict the port to hold, so substitution through a squatter now depends on a chance collision. The core still hands the requested rather than the bound port to its internal clients (`lib.rs:274` and `:276`), and the blocklist is still unsigned and fetched over plain HTTP; both are core changes shared with WP3.

When the preferred SOCKS port is held by another process, `bind_preferring` binds an operating-system-chosen port and publishes it, and the extension correctly gives that port to the bridge. `start_http`, however, hands the requested port, not the bound one, to the relay and to the blocklist refresher. The refresher therefore sends its SOCKS5 CONNECT and its HTTP request to whatever holds the requested port. The list itself is fetched over plain `http://` (`parse_url` refuses `https`, stating that the core has no TLS stack), carries no signature, and is installed as soon as it parses. Even without a squatter, its integrity therefore rests on the cleartext path from the exit to the list's origin.

Affected: `redsocket-http-core/src/lib.rs:253-260`, `:274`, `:276`, `start_http`; `src/blocklist.rs:467-476`, `install_json` ↪ ; `:713-799`, `spawn_refresh`, `refresh_loop`, `fetch_once`; `:801-806`, `parse_url`; `:925-960`, `socks_connect`. Also affects WP3.

Proof of Concept: In T15 an independent Python process (standing in for a co-resident app) held 127.0.0.1:20410 before the core was started with that port and a blocklist URL:

```
rs_start(20410) -> "ok:http:127.0.0.1:20401"; rs_socks_port=57696
[squatter] conn: greeting ver=5 methods=00
[squatter] CONNECT cmd=1 to lists.example.test:80
[squatter] request: GET /v1.json HTTP/1.1\r\nHost: lists.example.test\r\n...User-Agent: okhttp/4.12.0
[squatter] served forged blocklist (86 bytes)
rs_get_block_stats -> {"armed":false,...,"domains":1,"enabled":false,"list_version":4242,...}
CONNECT forged-block.example.test:443 with blocking OFF -> reply 05000001000000000000
CONNECT forged-block.example.test:443 with blocking ON -> reply 05020001000000000000
```

Impact: An application that binds the extension's preferred port before the tunnel starts receives the core's list request, which also signals the start of a RostamVPN session, and substitutes the list. A substituted list can hold up to 200,000 rules, so an attacker can censor arbitrary destinations through the user's own protection feature as soon as on-device filtering is enabled. A party on the path from the exit to the list origin can do the same without being on the device. In the RostamVPN brand as configured (the `classic` composition), no UI enables on-device filtering, so a substituted list is installed but not enforced. Enforcement follows as soon as the stored Guardian switch is set (`RedSocketPacketTunnelProvider.swift:247-256`).

Preconditions: The signed configuration enables the blocklist with a URL. Substitution by an app additionally requires that it holds the port before the tunnel starts; enforcement requires on-device filtering to be enabled.

Recommendation: Pass the bound port:

```
probe::start(bound_port);
blocklist::spawn_refresh(bound_port, &cfg.blocklist);
```

Sign the list with the publisher key, including a monotonic version and an expiry, and verify the signature before `install_json`. Fetch the list over TLS through the tunnel.

Classification: CWE-923 · MASVS-PLATFORM-1 · CVSS:4.0/AV:L/AC:L/AT:P/PR:L/UI:N/VC:L/VI:L/VA:L/SC:N/SI:N/SA:N

RVPN-01-413 WP4: WGTCP reports connected before any WireGuard handshake (Medium)

The WGTCP carrier sets its "up" flag as soon as the TCP connection to the server succeeds, before `serve` has even sent the handshake initiation. `start_tunnel` polls that flag and returns success, and the FFI then marks the core as running with status "connected". The provider installs the network settings and completes `startTunnel`, and the `auto` path never falls back to RedSocket. The SOCKS front-end also acknowledges a CONNECT with success before the inner TCP session exists. The health monitor consults the same running flag and the local port, so it never fires.

Affected: `vendor/redsocket_wgtcp/src/carrier.rs:93–118`, `run`, and `:131–137`, `serve`; `wgtcp-android/src/lib.rs:195–234`, `start_tunnel`; `wgtcp-android/src/ffi.rs:49–70`, `wgtcp_start`; `RedSocketTunnel/RedSocketPacketTunnelProvider.swift:508–532`, `:704–707`. Also affects WP5.

Proof of Concept: In T16, a loopback listener that accepts TCP and never answers was configured as the WGTCP endpoint:

```
wgtcp_start -> "ok" after 210 ms; wgtcp_status="connected"; wgtcp_is_running=1
CONNECT by IPv4 192.0.2.10:80 -> reply 0500000100000000000 after 0 ms
SOCKS reply 0500... then request sent; bytes back within 10 s: Err(WouldBlock) after 10001 ms
25 s later: wgtcp_status="connected"; wgtcp_is_running=1; local SOCKS port accepting=true
[silent] 156 bytes at +0 s, 312 at +5 s, 468 at +10 s (handshake initiations, never answered)
```

Impact: Any reachable host that is not the configured peer produces a permanent "VPN On" with all traffic dropped and no fallback to the working transport. That includes a censor-operated TCP endpoint, a middlebox that accepts and discards, and a revoked key. Combined with RVPN-01-407, the same disconnect between readiness and handshake hides an interception from the user.

Preconditions: WGTCP enabled by the signed policy, and an endpoint that accepts TCP without completing WireGuard.

Recommendation: Treat the carrier as up only after the first completed handshake, using boringtun's `Tunn::stats()` (whose first element is the time since the last handshake), and return an error after a timeout so that `auto` falls back. Export the handshake age through the FFI for the health monitor, and send the SOCKS success only once the inner TCP connection is established.

```
// sketch: WgPeer exposes its boringtun Tunn
let (since_handshake, ..) = self.peer.tunn().stats();
if since_handshake.is_some() { CARRIER_UP.store(true, Ordering::Relaxed); }
```

Classification: CWE-754 · MASVS-NETWORK-1 · CVSS:4.0/AV:N/AC:L/AT:P/PR:N/UI:N/VC:N/VI:L/VA:H/SC:N/SI:N/SA:N

RVPN-01-414 WP4: WGTCP resolves destination names outside the tunnel (High)

The bridge runs with its mapped-DNS module enabled, so applications resolve names to synthetic addresses and the bridge sends the original host name in its SOCKS5 request. The RedSocket core forwards that name to the exit inside the Noise session; T14 recorded no resolver call on that path. The WGTCP core does not forward the name. Its SOCKS front-end resolves every domain-type request with `tokio::net::lookup_host`, the operating-system resolver, inside the extension process and before anything enters WireGuard. Network Extension routes the provider's own connections outside its tunnel, so the lookup is expected to go in cleartext to the access network's resolver. If the platform instead applied the tunnel's DNS server to these lookups, the bridge would answer with a synthetic address and every name-based WGTCP connection would fail. Which of the two happens on iOS was not determined, because device testing was excluded; the first outcome is a leak and the second a functional failure.

Affected: `vendor/redsocket_wgtcp/src/socks5.rs:118–127`, `handle`, and `:166–181`, `resolve`; `HevSocks5Tunnel.swift:69–78` (mapped DNS enabled). Also affects WP5.

Proof of Concept: T16 loaded a `getaddrinfo` interposer into the WGTCP harness and issued one CONNECT by name. The interposer answered `.invalid` locally, so no query left the host:

```
CONNECT by name wp4-dns-probe.invalid:80 -> reply 0504000100000000000 after 1 ms
[gai] getaddrinfo(node=wp4-dns-probe.invalid, service=(null))
```

The same probe through the RedSocket core (T14) produced no `getaddrinfo` call; the name reached the exit inside the tunnel instead.

Impact: With WGTCP carrying traffic, the name of every destination the user visits would be disclosed in cleartext to the local network's resolver and attributed to the subscriber, while the UI shows the VPN on. That is deanonymisation of browsing, and it also hands the censor DNS-based blocking.

Preconditions: `wgtcp` present in `transport_order`; on the device, provider lookups resolved through the physical network, which is expected but not device-verified.

Recommendation: Never resolve locally in the WGTCP path. Resolve inside the tunnel: send the DNS query through the smoltcp stack to a resolver reachable only through WireGuard (for example at the profile's `gateway`), or carry names to the server. Add a test asserting that the WGTCP SOCKS path never calls the system resolver.

```
Target::Domain(name, p) => match tunnel_dns.resolve(name, want_v6).await { // via smoltcp to cfg.gateway
   Some(a) => (a, *p),
   None => { reply(&mut sock, REP_HOST_UNREACHABLE).await?; return Err(anyhow!("no answer")) }
},
```

Classification: CWE-319 · MASVS-NETWORK-1 · CVSS:4.0/AV:N/AC:L/AT:P/PR:N/UI:N/VC:H/VI:N/VA:N/SC:N/SI:N/SA:N

Miscellaneous Issues

RVPN-01-403 WP4: Logs recorded session metadata in the unified log and the shared container (Medium)

Fix Note: This issue has been fixed by the development team in commits 7b72411 and 3ba2f29 and verified by Red Team Ltd to be working as expected. Commit 7b72411 deleted the unreachable log viewer, the export and its all-zero key. Since 3ba2f29, `extLog` writes only through `os_log` with `%{private}@`, the provider deletes any old `extension.log` when it initialises, the egress address is no longer logged, and the app's `vpnLog` and the store's `payLog` mark their messages private (T27). The public `NSLog` lines that remain carry status only; the bridge's configuration dump (`HevSocks5Tunnel.swift:113` at 1b196be) still prints the loopback port and the tunnel addresses.

Note: Messages logged through `extLog`, `vpnLog` and `payLog` now appear redacted in Console and in sysdiagnose archives.

`extLog` writes every extension message three ways: `NSLog`, `os_log` with `%{public}@` (never redacted), and an append to `extension.log` in the App Group container. The file is truncated when the provider initialises, but has no size cap, no backup exclusion and default file protection. Lines carry ISO-8601 timestamps and include the exit endpoint label (`RedSocket: connected via http:host:port`), the egress address reported by the post-connect echo request (`SOCKS5 EXIT IP`), configuration size, protocol choice, WGTCP results, entitlement verdicts and health-check outcomes. At every appearance the app logs through `NSLog` the inputs of its region decision: a keyboard-derived region (IR or RU), the storefront country, locale and time zone. It also logs `cid` persistence events and the purchase tier, product and paid-until date. The Rust cores install no logger on iOS, so their log statements produce no output. The in-app log viewer and its "encrypted" export, which used an all-zero AES-256-GCM key, are unreachable in bd9bbc6: `showLog` is never set to `true`, and `onExportLog` is never invoked. That code is dead but still compiled.

Affected: `RedSocketTunnel/RedSocketPacketTunnelProvider.swift:26–59, :571, :777`; `rostamvpn/RostamVPNApp.swift :695–696, :1821–1826, :1852–1896`; `rostamvpn/RegionPolicy.swift:24–35`; `rostamvpn/ConfigPolicy.swift:376–385`; `rostamvpn/CashierIdentity.swift:15–23`; `rostamvpn/Store.swift:293–297`.

Proof of Concept: Static (T19 locates the `%{public}` call site):

```
// RedSocketPacketTunnelProvider.swift:44–45, 777
NSLog("[Rostam] %@", msg)
os_log("%{public}@", log: extensionLog, type: .default, msg)
self?.extLog("SOCKS5 EXIT IP: \(ip)")
```

Impact: Anyone holding a sysdiagnose from a seized, unlocked device, or the App Group container from a backup, obtains a timeline of tunnel sessions with exit endpoints and egress addresses, together with signals of the user's language and location.

Recommendation: In release builds, log endpoints, addresses and region inputs only as private or not at all:

```
private let log = Logger(subsystem: "com.rostamvpn.redsocket", category: "tunnel")
log.info("connected via \(carrier, privacy: .private(mask: .hash))")
```

Cap and rotate `extension.log` and exclude it from backup. Drop the region and purchase lines from release builds, and delete the dead export code and its key.

Classification: CWE-532 · MASVS-STORAGE-2 · CVSS:4.0/AV:P/AC:L/AT:P/PR:N/UI:N/VC:H/VI:N/VA:N/SC:N/SI:N/SA:N

RVPN-01-404 WP4: Identity, WireGuard key and nationality signals kept in backed-up storage (Medium)

Fix Note: This issue has been partially fixed by the development team in commits bc011b9, 1b196be, fe379aa and 35dad8d and verified by Red Team Ltd to be working as expected for the keychain, the device signals and the Guardian history. `KeychainService` now stores items with `kSecAttrAccessibleAfterFirstUnlockThisDeviceOnly` and moves older items to that class with `SecItemUpdate` when it reads them; the read query, as amended in 1b196be, matches items of any class, so items written under the earlier class are found and moved. `captureDeviceSignals` records keyboard languages and device type only after relay consent and deletes them otherwise, and `IPGuardStore` sets `isExcludedFromBackup` after every write (T27, T31). Open: the WireGuard private key is still copied into the App Group defaults (`RostamVPNApp.swift:1465` at 1b196be), and those defaults, which also hold the `cid` and the decrypted configuration, are still included in backups.

Note: Keychain items, including the client identifier, no longer restore from a backup to a new device, so the app on a replacement device starts with a new identifier.

`KeychainService` stores every item with `kSecAttrAccessibleAfterFirstUnlock`, without the `ThisDeviceOnly` suffix. That applies to the `cid` and, once WGTCP is enabled, to `wgtcp_private_key` and `wgtcp_public_key`. Items in this class are eligible for restoration to another device from an encrypted backup. Because the extension has no keychain access group, the app copies the WireGuard private key into the App Group defaults as part of the `wgtcpConfig` JSON, and caches the server profile alongside it. The App Group defaults also hold the `cid`, the decrypted configuration (endpoints and server key), `entitledRegion`, `isFreedomRegion`, usage counters and the control cursor. They further hold the user's keyboard languages and device type, which `PeerActivation.captureDeviceSignals()` records at every appearance whether or not the relay was ever consented to. App Group container data is part of device backups, and nothing in the tree sets `isExcludedFromBackup` (T19: zero matches).

Affected: `rostamvpn/KeychainService.swift:13–25`; `rostamvpn/WgtcpProvisioner.swift:18–37, :89–114`; `rostamvpn/RostamVPNApp.swift:699–708, :1534–1549`; `LocalPackages/rostam-apple/Sources/RostamKit/PeerActivation.swift:44–58`; `Config/Tunnel.entitlements`.

Proof of Concept: Static:

```
// KeychainService.swift:22
query[kSecAttrAccessible as String] = kSecAttrAccessibleAfterFirstUnlock
// RostamVPNApp.swift:1538–1539
if let cfg = WgtcpProvisioner.config() {
   UserDefaults(suiteName: suiteName)?.set(cfg, forKey: "wgtcpConfig") // contains private_key
```

Impact: A seized device or a backup yields a persistent identifier that survives reinstall and that the operator links to purchases and sessions. It also yields the WireGuard private key, which allows impersonating the device to the WGTCP service, the endpoint list, and a nationality signal (installed keyboard languages).

Recommendation: Use `kSecAttrAccessibleAfterFirstUnlockThisDeviceOnly`. Keep the WireGuard key in the keychain under an access group the extension can read; App Group identifiers are valid keychain access groups. Stop copying the key into defaults, exclude App Group files from backup, and record keyboard signals only after relay consent.

```
query[kSecAttrAccessible as String] = kSecAttrAccessibleAfterFirstUnlockThisDeviceOnly
query[kSecAttrAccessGroup as String] = BrandRuntime.appGroup
```

Classification: CWE-922 · MASVS-STORAGE-1 · CVSS:4.0/AV:P/AC:L/AT:P/PR:N/UI:N/VC:H/VI:L/VA:N/SC:N/SI:N/SA:N

RVPN-01-405 WP4: Relay consent gate lacks a consent flow and an effective withdrawal (Low)

Fix Note: This issue has been partially fixed by the development team in commits 7a0e32f and fe379aa and verified by Red Team Ltd to be working as expected for withdrawal. The provider's relay timer no longer cancels

itself after an activation; it calls `PeerActivation.maybeActivate` every five seconds, which re-reads the opt-in and disables a running relay once consent is withdrawn, and device signals are recorded only after consent (RVPN-01-404; T27). Open: there is still no consent flow, the "Help Improve" switch still controls nothing while its footer promises diagnostics (`IPAccountTab.swift:150–153`), and the core's relay still resolves names in cleartext and announces itself in its handshake.

The relay lets the device act as a SOCKS exit for targets chosen by the operator's gateway. It is gated by the signed `probe` policy, by `PeerActivation.userOptIn` and by sampling, region, keyboard and device gates. In bd9bbc6 nothing writes `userOptIn` (T19: the setter has no caller), so the relay cannot start, and the gate holds. Four defects remain, and each would become exposure the moment a consent toggle is added. First, there is no consent screen. Second, withdrawal does not reach a running relay: the provider cancels its relay timer as soon as activation succeeds, and the setter's `deactivate()` runs in the app process, which hosts its own copy of the core. That contradicts the README's statement that turning the opt-in off disarms a running session. Third, the visible "Help Improve" toggle only reaches `probeEnabled` in the provider configuration, which the extension merely logs, although the `tabs` composition describes it as sending anonymous connection diagnostics. Fourth, once armed, the relay dials targets from the device's real address, resolves names by cleartext UDP through a hard-coded third-party public resolver when no system resolver file is readable, and announces its identity (`rostam_` followed by the `cid`) and the time zone in a cleartext handshake.

Affected: `LocalPackages/rostam-apple/Sources/RostamKit/PeerActivation.swift:31–40, :166–222`; `RedSocketTunnel/RedSocketPacketTunnelProvider.swift:84–92, :670–680`; `rostamvpn/RostamVPNApp.swift:848–851, :2016–2028`; `rostamvpn/Tabs/IPAccountTab.swift:150–153`; `redsocket-http-core/src/peer.rs:390–421, :585–616, :700–703`.

Proof of Concept: Static:

```
// RedSocketPacketTunnelProvider.swift:674–678 (timer ends after the first activation)
relayTimer.setEventHandler {
   if PeerActivation.maybeActivate(socksPort: relaySocksPort) { relayTimer.cancel() }
}
```

Impact: None in this build. The risk is regression into deanonymising behaviour without informed consent, together with a privacy control that does not do what it says.

Recommendation: Remove the relay from iOS builds until a consent flow exists. If it stays, make one consent screen the only writer of the opt-in, send an explicit provider message that calls `rs_disable_peer` inside the extension, keep re-checking consent while armed, resolve names through the tunnel, and remove or truthfully relabel "Help Improve".

Classification: CWE-451 · MASVS-PRIVACY-4 · CVSS:4.0/AV:N/AC:H/AT:P/PR:N/UI:N/VC:L/VI:N/VA:N/SC:N/SI:N/SA:N

RVPN-01-408 WP4: Signed configuration lacks freshness and rollback protection (Medium)

`cfgseal::unseal` verifies an Ed25519 signature over the JSON body and returns it. It has no notion of version, issue time, expiry or audience. `RemoteConfig` stores any document that opens and lists at least one RedSocket entry, even if it is older than the stored one, and it displays the version from the unsigned `X-Config-Version` header. A single 404 on the v2 path pins the client to v1 permanently. The QR bootstrap prepends scanned entries to the stored list without deduplication or a cap, and because the seal is fleet-wide, codes minted for any brand of the fleet verify. The symmetric layer of the seal gives no confidentiality: its key is reconstructed from constants in the public source (T18), and a single Noise server key is distributed per document.

Affected: `redsocket-http-core/src/cfgseal.rs:73–114`; `rostamvpn/RemoteConfig.swift:49–62, :87–111, :119–129`; `rostamvpn/QrConfigView.swift:263–336`; `RedSocketTunnel/RedSocketPacketTunnelProvider.swift:556–558`.

Proof of Concept: T18 opened the publisher-sealed test vector shipped in `cfgseal.rs` with the core's own example tool and the constant from `BrandRuntime.swift`. It returned a 92-byte document with only `version` ("`rt-test`") and `transport`, and no time fields:

```
unseal-config sealed-test-vector.b64 "$FLEET_CONSTANT" -> exit 0; keys ['transport', 'version']
```

Nothing in the client could have refused it as stale.

Impact: An adversary who can serve the configuration response, through TLS interception or a compromised cashier, or who distributes QR codes through censor-run "bootstrap" channels, can replay any previously signed document. That puts retired or seized endpoints first, reverts `transport_order` (for example to enable WGTCP), or restores old feature blocks. With a seized POP holding the shared key, a replay steers users to censor-operated exits. Captured or published codes also enumerate endpoints for blocking.

Recommendation: Sign a monotonic integer `version`, `issued_at` and `expires_at`. Reject documents that are older than the stored one or expired, and display the signed version. Deduplicate and cap QR entries, treat a 404 as transient, give each POP its own key with revocation carried in the document, and describe the symmetric layer as obfuscation.

```
guard let v = obj["version"] as? Int, v > storedVersion,
    let exp = obj["expires_at"] as? Double, exp > Date().timeIntervalSince1970
else { return done(false, false) }
```

Classification: CWE-294 · MASVS-CODE-4 · CVSS:4.0/AV:N/AC:H/AT:P/PR:N/UI:P/VC:L/VI:L/VA:L/SC:N/SI:N/SA:N

RVPN-01-409 WP4: Exit control-stream notes change client state without further authentication (Low)

Fix Note: This issue has been partially fixed by the development team in commit a372d28 and verified by Red Team Ltd to be working as expected for future-dated echoes: `GuardianMessage.updatedSeconds` now clamps an echo's `updated` time to the present, and `GuardianMessageTests` covers the clamp (T27, T28). Open: an echo without `updated` still replaces the user's lists, an `entitlement` note with state `paid` still sets `isPaid`, and notes are still authenticated only by the exit session.

When the signed configuration enables `protection` or `messaging` and MUX2 is on (the core's test suite asserts that the shipped configuration enables MUX2), the exit can open a control stream and push JSON notes. The core queues them unmodified: up to 200 notes of up to 65,535 bytes each. The extension drains the queue every five seconds and acts on three note types. An `entitlement` note with state `paid` sets `isPaid` for every brand. A `rules` note replaces the user's allow and deny lists whenever its `updated` field is absent or newer than the local one; the exit controls that field, so it can always make its lists win, and they are persisted. A `cursor` note is persisted and injected into the next configuration load. Other notes, such as `notice` with an HTTPS link, are queued for the app. The user's own lists are uploaded to the exit in every capped session. Notes are authenticated only by the Noise session with the exit.

Affected: `RedSocketTunnel/RedSocketPacketTunnelProvider.swift:214–236, :334–385, :413–422; redsocket-http-core` ↪ `/src/ctl.rs:30–41, :385–437; src/mux.rs:181–182.`

Proof of Concept: Static. The extension applies the rules echo as follows:

```
// RedSocketPacketTunnelProvider.swift:224–235
if let e = echoUpdated, e < localUpdated { return } // absent or future 'updated' always applies
_ = core.setUserRules(json: json)
g?.set(core.userRulesJSON(), forKey: GuardianMessage.rulesKey)
```

Impact: A compromised exit, or anyone holding the shared exit key, can grant the paid tier and persistently rewrite the user's filter lists, which on `tabs` brands amounts to censorship through the user's own protection feature. It can also queue about 13 MB of notes inside the memory-limited extension. For the `classic` RostamVPN brand the practical effect is limited to entitlement.

Recommendation: Sign entitlement and rules notes with a key the client already trusts, bound to the `cid` and a timestamp. Clamp `updated` to the current time, cap the queue by bytes, and disclose the rules upload in the UI.

Classification: CWE-807 · MASVS-CODE-4 · CVSS:4.0/AV:N/AC:H/AT:P/PR:H/UI:N/VC:L/VI:L/VA:N/SC:N/SI:N/SA:N

RVPN-01-415 WP4: Shipped RedSocketCore did not match the published source (Low)

Fix Note: This issue has been fixed by the development team in commit 10433da, the amended form of b560865, and verified by Red Team Ltd to be working as expected. The commit replaced all four core slices

with archives built from rostamvpn-android commit 621ba38, whose three core subtrees are byte-identical to the f60abd3 sources reviewed here, and recorded that revision and the neutral build directory in `README.md`. Rebuilt from those sources with the documented path mapping, both RedSocketCore slices reproduced bit for bit (identical SHA-256), and both WgtcpCore slices function for function (259 of 259 and 249 of 249 functions, all 484 dependency members byte-identical) (T26). The slices at 1b196be carry the same SHA-256 values (T27). The rest of the recommendation remains open: no revision in the frameworks' `Info.plist`, no attestation or SBOM, and the unchanged bridge archive still comes from an unpinned upstream.

The repository commits both transport cores and the bridge as opaque archives, with no record of the source revision or build environment. Red Team Ltd rebuilt both cores from f60abd3 with the toolchain the archives identify (Rust 1.91.1, whose standard-library hash matches). The WgtcpCore archives matched once the build path was reproduced: all 484 dependency members were byte-identical, and both first-party objects contain the same functions with the same sizes (259 of 259 and 249 of 249). The RedSocketCore archives did not match. The shipped core was compiled from the crates.io registry cache with the home directory remapped to `x`, not from the vendored tree under the published remap rule; with that configuration reproduced, 467 of 469 members were identical. The crate's own object differs in five functions, and `socks5::Upstream::active_label` exists only in the shipped binary. The hev bridge is built from upstream HEAD without a pinned revision and carries no version marker.

Affected: `LocalPackages/rostam-apple/RedSocketCore.xcframework` (both slices), `Info.plist` files of both frameworks; `LocalPackages/hev-socks5-tunnel-apple/build-ios.sh:26-31`; `README.md`, Dependencies section.

Proof of Concept: T06 and T08:

```
RedSocketCore ios-arm64: shipped 33,996,992 bytes, rebuilt 33,997,376; members identical 467/469
functions: shipped=708 rebuilt=707; differing in presence or size: 5
  mux::MuxPool::pick closure 6352/6340, noise::other 224/292, reach::run_loop closure 3140/3120,
  protect::protected_connect closure 1472/1560, socks5::Upstream::active_label 164/absent
WgtcpCore ios-arm64 (shipped prefix reproduced): common members 485, identical 484 (symbol index differs)
```

Impact: Readers of the public repository cannot verify that the shipped transport is the reviewed source; this report's core analysis rests on f60abd3. A compromised build host, or an upstream change in the unpinned bridge, would not be visible in the repository.

Recommendation: Build the cores and the bridge in CI from pinned revisions and vendored sources, with a path remap that excludes user paths (`--remap-path-prefix=$PWD=.`). Record the revision, lock hash, toolchain and SDK in an SLSA v1.0 provenance statement and in each framework's `Info.plist`. Publish a CycloneDX 1.7 or SPDX 3.0.1 SBOM, add a rebuild-and-compare job, and pin hev-socks5-tunnel and its lwIP revision.

Classification: CWE-1357 · MASVS-CODE-3 · CVSS:4.0/AV:N/AC:H/AT:P/PR:H/UI:N/VC:L/VI:L/VA:N/SC:N/SI:N/SA:N

RVPN-01-416 WP4: Native build layer hygiene and stale interfaces (Info)

Fix Note: This issue has been partially fixed by the development team in commit 0af77e2 and verified by Red Team Ltd to be working as expected for item 2 and the archive in item 3: the stale header `hev_socks5_tunnel.h` and the unlinked `libyaml.a` are gone (T27, T28). The vendored libyaml source tree (item 3) and items 1 and 4 to 9 remain open. A clean simulator build of the remediated tree succeeded only with a bridge archive converted to the simulator platform outside the repository (T29).

Several build-level defects were observed that carry no direct exploit but undermine maintainability and verification. Items 4 and 5 are unchanged in the archives that 10433da introduced.

1. The bridge archive is device-only, yet `Package.swift` links it for every platform; `build-ios.sh` produces a simulator variant but never commits or references it. A simulator build therefore fails at the extension link (T23). The script also still passes `-fembed-bitcode`.
2. `hev_socks5_tunnel.h` declares `hev_socks5_tunnel_get_stats` and `hev_socks5_tunnel_set_tunnel_fd`, which the archive does not export, and a one-argument `hev_socks5_tunnel_main` that conflicts with the library's two-argument function. The compiler warns that the umbrella header does not include it.
3. `libyaml.a` and the 68-file vendored libyaml tree are unused; libyaml 0.2.5 is linked inside the bridge archive.

4. The 24 ring objects in each WgtcpCore slice target iOS 27.0 with SDK 27.0, while the app targets iOS 15.6. The deployment target was not set for the cargo build.
5. RedSocketCore exports 34 JNI entry points, and `jni` and `android_logger` are linked into both iOS cores.
6. `chacha20` 0.10.1, a yanked release, is locked through `rand` 0.10.2.
7. The SwiftLint phase executes whatever `swiftlint` is first on `PATH`, with user-script sandboxing disabled for the app target.
8. Swift language mode 5.0 hides data races, such as `activeTransport` and `Tun2SocksWrapper.isRunning` being touched from several queues, that Swift 6 mode would diagnose.
9. Thirteen core tests depend on generated files outside the crates.

Affected: `LocalPackages/hev-socks5-tunnel-apple/Package.swift:31-41`; `build-ios.sh:44, :67-75`; `Sources/CHevSocks5Tunnel/hev_socks5_tunnel.h:8-23`; `rostamvpn.xcodeproj/project.pbxproj:394, :434`; `redsocket-http-core/Cargo.toml` and `wgtcp-android/Cargo.toml` dependency sections.

Proof of Concept: T23 produced `ld: building for 'iOS-simulator', but linking in object file (...libhev-socks5-tunnel.a[2](hev-rbtree.o))built for 'iOS'`. T02 listed the 24 objects with `minos 27.0`, T05 the missing exports, and T12 and T13 the yanked crate and its path.

Recommendation: Ship the bridge as an xcframework with device and simulator slices, delete the stale header and the unused libyaml copies, and set `IPHONEOS_DEPLOYMENT_TARGET` for cargo builds. Gate `jni` and `android_logger` behind `cfg(target_os = "android")`, re-resolve `rand` and `chacha20`, invoke SwiftLint through a pinned plugin or skip it in release builds, and move to Swift 6 language mode.

Classification: CWE-1104 · MASVS-CODE-3 · CVSS:4.0/AV:L/AC:H/AT:P/PR:H/UI:N/VC:N/VI:L/VA:L/SC:N/SI:N/SA:N

RVPN-01-417 WP4: Live identifiers in the published trees (Info)

Fix Note: This issue has been fixed by the development team in commits dcb2d05 and 10433da (the amended form of b560865) and verified by Red Team Ltd to be working as expected for items 1 and 2. After dcb2d05 every address in the fixture (lines 62, 64 and 80) lies in RFC 5737 documentation space (T24). After 10433da none of the four core slices contains the build host's account name, a home-directory path or the workspace name, and the embedded prefixes are neutral (`/build/rostamvpn-android/...` and `/rustup/...`) (T25, T27). Items 3 to 5 remain by design: the resolver address is a functional constant of the relay code, the test vector belongs to the WP3 tree, and the mailboxes are upstream attribution.

The publication scrub missed several identifier classes. Following the engagement's redaction rule, they are named here by location and class only; the values were handed to the engagement lead for scrubbing.

1. `rostamvpn/Tabs/IPGuardian.swift`, lines 62 and 80: a public IPv4 address outside the documentation ranges, inside the `#if DEBUG` fixture (lines 42 to 119). It is not compiled into Release builds but is published in source.
2. `LocalPackages/rostam-apple/WgtcpCore.xcframework`, both slices: the build host's account name and private workspace layout, in about 534 path strings per archive. The published remap rule replaces only the first path component, and these strings reach every app binary through panic locations.
3. `LocalPackages/rostam-apple/RedSocketCore.xcframework`, both slices: a hard-coded third-party public DNS resolver address. This is the relay fallback discussed in RVPN-01-405 and is functional, not project infrastructure.
4. Also affects WP3: the sealed test vector in `redsocket-http-core/src/cfgseal.rs`, line 166, decrypts with constants in the same file to a configuration naming a public IPv4 endpoint (T18). Sealing does not conceal it.
5. Third-party author mailboxes in vendored files (`hev-main.h` line 4, libyaml documentation, OpenSSL attribution strings inside the ring objects). These are upstream attribution, and no action is recommended.

Affected: As listed.

Proof of Concept: T20 scanned every text and binary file of the tree for public IPv4 addresses outside RFC 5737, mailboxes, home-directory paths, CDN distribution names, OAuth client identifiers and long hexadecimal values. T03 extracted the embedded path strings; T18 decrypted the test vector.

Impact: The fixture and the decrypted test vector publish addresses that a censor can block, and the build paths disclose an engineer's account name. No user data is affected.

Recommendation: Replace the fixture address with one from 192.0.2.0/24. Rebuild WgtcpCore with a remap that covers the whole workspace path, re-seal the test vector with documentation addresses, and add the T20 checks to the publication script.

Classification: CWE-540 · MASVS-STORAGE-2 · CVSS:4.0/AV:N/AC:L/AT:N/PR:N/UI:N/VC:L/VI:N/VA:N/SC:N/SI:N/SA:N

RVPN-01-418 WP4: Personal mailbox used as the support contact (Info)

Fix Note: This issue has been fixed by the development team in commit bd9bbc6 and verified by Red Team Ltd to be working as expected.

The brand definition's `BRAND_SUPPORT_EMAIL` is substituted into the app's Info.plist as `BrandSupportEmail`. It is shown in the About screen and in the contact section of the in-app privacy policy, so every installation and the public repository carried it. Before the fix the value was an individual's mailbox. For the people behind a censorship-circumvention service, a personal address is a lead for targeting and social engineering. Commit bd9bbc6 ("Use a role mailbox for the support contact") replaced it with a role address at the product's domain.

Affected: `Brands/rostam/Brand.xcconfig:35`; `Config/RostamVPN-Info.plist:43-44`; `LocalPackages/rostam-apple/Sources/RostamKit/BrandRuntime.swift:99, :113-118`; `rostamvpn/RostamVPNApp.swift:2249-2257`.

Proof of Concept: Red Team Ltd verified the fixed state. `Brand.xcconfig` line 35 now holds a role mailbox, and the T20 scan found no other mailbox in any first-party file; the remaining matches were image file names with `@2x` and `@3x` suffixes and third-party attribution addresses. The commit description was read from the repository's commit-message file. Earlier commits remain part of the public history and were not examined, because the rules for this work package exclude running git.

Impact: Exposure of an individual's contact address; no effect on users.

Recommendation: Keep role addresses in all brand files and consider whether the earlier value in the public history warrants a rewrite.

Classification: CWE-359 · MASVS-STORAGE-2 · CVSS:4.0/AV:N/AC:L/AT:N/PR:N/UI:N/VC:L/VI:N/VA:N/SC:N/SI:N/SA:N

Conclusions

Red Team Ltd assessed the RostamVPN iOS application and its packet tunnel extension at commit bd9bbc6 against a demanding threat model: a state-level censor with national DPI, hostile networks, coerced trust anchors, device seizure and co-resident malicious applications. Version 1.1 re-verifies the maintainers' remediation on main up to commit 1b196be. At bd9bbc6 the client rested on sound foundations that its integration layer let down: the cryptographic choices are well made, the configuration is signed and verified, and the FFI boundary is carefully built, but the way the extension routed, monitored and logged traffic left paths through which a user's real address, destinations and identity could escape. The remediation closed the most direct of those paths, IPv6 and the logs, and narrowed most of the others. What remains is concentrated in the failure mode, the transport signatures and the WGTCP path. The table records the status of every issue; the paragraphs that follow walk through each area in the order of the brief and describe the state at 1b196be.

Issue	Severity	Status at 1b196be	Verified commits
RVPN-01-401	High	Fixed	1131ebf
RVPN-01-402	High	Partially fixed	9c0f48b
RVPN-01-403	Medium	Fixed	7b72411, 3ba2f29
RVPN-01-404	Medium	Partially fixed	bc011b9, 1b196be, fe379aa, 35dad8d
RVPN-01-405	Low	Partially fixed	7a0e32f, fe379aa
RVPN-01-406	High	Partially fixed	ebd4fc7
RVPN-01-407	Medium	Partially fixed	90c3578
RVPN-01-408	Medium	Open	none

Issue	Severity	Status at 1b196be	Verified commits
RVPN-01-409	Low	Partially fixed	a372d28
RVPN-01-410	High	Open; Client Note	none
RVPN-01-411	Medium	Partially fixed	bf88011
RVPN-01-412	Medium	Partially fixed	bf88011
RVPN-01-413	Medium	Open	none
RVPN-01-414	High	Open	none
RVPN-01-415	Low	Fixed; provenance items open	10433da
RVPN-01-416	Info	Partially fixed	0af77e2
RVPN-01-417	Info	Fixed; remaining items by design	dcb2d05, 10433da
RVPN-01-418	Info	Fixed	bd9bbc6

Tunnel network settings. The provider's network settings, the profile the app creates and the bridge configuration were read in full, at bd9bbc6 and again at 1b196be (T19, T27). Much of this was right from the start. The tunnel claims the IPv4 default route with an address inside 198.18.0.0/15, a benchmark range unlikely to collide with real networks. The resolver is the bridge's mapped-DNS module at 198.18.0.2, with a 100.64.0.0/10 pool and a 10,000-entry cache, and the MTU of 1500 is consistent between the tunnel settings and the bridge. On the default RedSocket path, names never become cleartext queries: the bridge returns synthetic addresses, the host name travels to the exit inside the Noise session, and the resolver interposer in T14 recorded no lookups. UDP either rides inside MUX2 through SOCKS5 UDP ASSOCIATE or is declined, so the analysed code offers no UDP escape route. The gap was IPv6, which bd9bbc6 did not claim at all (RVPN-01-401). Since 1131ebf the settings live in RostamKit's `TunnelNetwork`, which claims `::/0` with a ULA address next to the IPv4 default route and makes the tunnel resolver authoritative for every domain; a unit test guards both (T28). IPv6 destinations now travel to the exit, where they fail if the exit has no IPv6. The failure mode has not changed: the profile sets neither `includeAllNetworks` nor `enforceRoutes` nor any on-demand rule, which the maintainers describe as a product decision (RVPN-01-402). One consequence lies outside this work package: the bridge forwards every destination it receives, including LAN, private and link-local addresses, through SOCKS to the exit. Whether the exits refuse such destinations is a question for WP1 and WP2 (Appendix D, question 4).

Tunnel state versus core state. The start sequence, transport fallback, health monitor, stop path, entitlement refresh and the app's state machine were traced, and the cores' readiness signals were exercised in T16 and T17. Start is well structured. The provider refuses to run without the App Group, enforces the entitlement gates itself rather than trusting the app, and records a refusal reason that the app can display. A WGTCP failure falls back to RedSocket unless WGTCP was explicitly chosen. The core bounds endpoint selection to 60 seconds, with a 32-second direct tier and fallbacks raced after 10 seconds, so a fully blocked network ends in a definite failure rather than an endless "Connecting". Health is where the design still fails. The extension's notion of health is local: a running flag and a listening port. T17 and T16 showed for both transports that these signals stay positive while every connection fails; when they do turn negative, the tunnel is cancelled and the device falls back to the physical network (RVPN-01-402). Since 9c0f48b, `stopTunnel` cancels the health timer and asks the bridge to quit, which clears its running flag once the bridge thread ends, so a reused provider process can start it again. There are still no `sleep`/`wake` handlers, no path monitor and no `reasserting` phase. The app's own state machine adds a further gap. It gives up after 30 seconds, while the core may take up to 90 seconds to deliver its verdict. A tunnel that completes after the app's timeout is then ignored by `handleVPNStatusChange`, which accepts `.connected` only while the UI is still connecting, so the app shows "VPN Off" while the system tunnel is up until the view next appears.

SOCKS5 bridge and local proxy. The generated bridge configuration, both cores' SOCKS servers and the port handling were reviewed and exercised (T14, T15, T30). The bridge configuration is small and generated entirely by the app, so the embedded libyaml parser sees no attacker-influenced input. The RedSocket core binds only to loopback and reports the port it actually bound, and the extension gives that port to the bridge. The tun file descriptor is located with the established `UTUN_OPT_IFNAME` technique. Since bf88011 the extension picks a random preferred port between 20000 and 49151 for each session. That removes the fixed-port signature and the predictable target for squatting (RVPN-01-411, RVPN-01-412). The proxy still serves every local process

without authentication and deviates from RFC 1928 method selection, and T30 showed that a loopback scan finds such a port in milliseconds. The core still hands the requested rather than the bound port to its internal clients, and the blocklist is still unsigned; with a random port, exploiting that now needs a collision rather than a squatter. Per-session credentials and a two-line change in `start_http` would close both.

Keychain and App Group storage of keys. Every keychain query, every App Group key and the file-protection and backup attributes were reviewed. Generation is sound. The `cid` comes from `SecRandomCopyBytes`, and the WireGuard key pair from the operating system's CSPRNG through `getrandom`, which the vendored test suite checks for distinct keys that are not the zero point. No item is marked synchronisable, so nothing enters iCloud Keychain, and the extension is granted only the packet-tunnel and App Group entitlements. Several storage weaknesses have now been closed. Since bc011b9 and 1b196be, keychain items are bound to the device, and older items move to the device-bound class when read. Since fe379aa, keyboard and device signals are recorded only after relay consent. Since 35dad8d, the Guardian history is excluded from backup on every write (T31). Two parts remain open (RVPN-01-404). The maintainers acknowledge that the WireGuard private key is still copied into App Group defaults, because the extension cannot read the app's keychain group. Those defaults, which also hold the `cid` and the decrypted configuration, are still part of device backups.

Identity and configuration exposure. The life cycle of the `cid` and of the stored configuration was traced across app, extension, cashier and exit. The `cid` is 128 bits of randomness and is not derived from any hardware or advertising identifier, and the app uses no IDFA, no IDFV and no App Tracking Transparency. On iOS the time-zone field of the exit hello stays empty unless the relay is armed, as T14 showed (`META||||` followed by the `cid`). The `cid` deliberately survives reinstallation, travels to the exit in every carrier hello, and reaches the cashier in query strings and registration bodies, the latter together with the device's time zone. Because the keychain item is now device-bound, a backup restored to a new device no longer carries it, and the app there starts with a new identifier. A copy of the identifier nevertheless remains in the App Group defaults, which are still backed up, so it should still be treated as a durable device identifier: minimise where it is sent and keep it out of URLs.

Signed configuration and rollback. The remote fetch, the QR bootstrap and the seal implementation were reviewed, and the publisher's test vector was opened in T18. The Ed25519 verification is sound and is the only gate: neither path accepts unsigned input. The QR flow rejects foreign, damaged and tampered codes with messages that tell the user why, and its camera usage string is accurate. The Noise server key comes only from the signed document, and App Transport Security runs with platform defaults and no exceptions. Freshness is still missing (RVPN-01-408, unchanged at 1b196be). The seal has no notion of version or time, its symmetric layer is readable with constants from the public source, the QR merge grows without bound, and a single 404 pins the client to the v1 path. With one Noise key per document, a seized POP plus a replayed document is a credible way to steer users to an adversary's exit, so freshness and per-POP keys should be designed together.

Control-stream notes from the exit. The capability negotiation, the core's control-stream server and the extension's note handling were traced. The design shows care. The control stream is served only when the signed policy announces capabilities, the core caps the queue at 200 notes and bounds frames by their 16-bit length, and cursor values more than a day ahead are rejected. A session that the exit has acknowledged as protected also refuses to fall back to the unmonitored legacy path after a MUX2 failure, so the exit's verdict cannot be routed around. Since a372d28 a rules echo can no longer date its lists into the future (T28). The weakness that remains is authentication of content (RVPN-01-409). An echo without `updated` still replaces the user's own lists, and an entitlement note still sets the paid flag, on the strength of the exit session alone. For the `classic` RostamVPN brand the effect is mostly commercial; brands that render the Guardian are more exposed.

Logging. All logging call sites in Swift and the logger initialisation in the cores were reviewed. The Rust cores are silent on iOS, because no logger is installed and release builds cap the log level at warning. At bd9bbc6 the Swift layer wrote exit endpoints, egress addresses and region inputs to the unified log as public strings and to an unbounded file in the shared container. Since 7b72411 and 3ba2f29 that is fixed (RVPN-01-403). The dead viewer, the export and its all-zero key are deleted, both processes log as private data, `extension.log` is removed, and the egress address is no longer logged at all. The public lines that remain carry status text only. The bridge's configuration dump, which shows the loopback port and the tunnel addresses, and the APNs token line, which cannot run without an entitlement, are the only ones worth tidying.

Direct control-plane requests and the API key. Every network call the app makes outside the tunnel, and its timing at launch, was traced again at 1b196be. The original design deferred cashier calls while the tunnel was

down, but at bd9bbc6 a launch-time race defeated it (RVPN-01-406). Since ebd4fc7, cashier calls wait until the saved profiles have been read and are then deferred while a profile exists and the tunnel is down, as intended. The rule lives in `ControlPlane.mustDefer`, which a unit test covers (T28). The configuration fetch at launch still goes direct, as the maintainers acknowledge, and so does WGTCP provisioning; a shared API key travels with every configuration request. For a service whose users are themselves the target of identification, routing those two requests through the tunnel or a fronted base is the remaining step. Two launch-time calls are dead in this build: `registerForRemoteNotifications`, without an `aps-environment` entitlement, and the background refresh task, whose identifier is not in a `BGTaskSchedulerPermittedIdentifiers` list (Appendix D, question 9).

Certificate pinning. Every `URLSession` configuration and trust decision was reviewed. App Transport Security is not weakened anywhere. StoreKit 2 is used correctly: transactions are verified through Apple's JWS and finished only after the cashier confirms, and a lapse reported by the cashier is not applied while StoreKit still holds a current subscription. No endpoint is pinned, however. The configuration does not need pinning thanks to its signature, but the WGTCP profile and the entitlement answers rely on platform TLS alone. Since 90c3578 a missing WGTCP policy flag means disallowed, which closes the default-allow path; the profile itself is still unsigned (RVPN-01-407).

Relay and probe consent gate. `PeerActivation`, the provider's relay timer, the core's relay and measurement modules and the UI toggles were reviewed. The relay is inert in this build. Nothing can set the opt-in, the opt-in and the signed policy both default to off, and the core refuses to relay unless explicitly enabled; reachability measurement starts only behind the same gate. Since 7a0e32f the provider keeps re-checking consent while the relay is armed, so a withdrawal now disarms a running relay; since fe379aa the device signals wait for consent. What still should not ship is the rest of the design (RVPN-01-405). There is no consent flow, the visible "Help Improve" switch still controls nothing, and once armed the relay would operate from the user's real address, with cleartext name resolution through a third-party resolver.

WGTCP readiness and transport. The carrier state machine, the FFI status, the SOCKS front-end and the record framing were reviewed and exercised (T16). The vendored WGTCP library's own test suite passed in full (T11), and boringtun and smoltcp are current, maintained components. The integration is unchanged at 1b196be and not ready for users in the threat model. It resolves names outside the tunnel (RVPN-01-414), reports success before WireGuard has spoken (RVPN-01-413), accepts an unsigned profile (RVPN-01-407) and opens every carrier with a constant 16-byte prefix (RVPN-01-410). The maintainers note that a handshake signal requires an export from the WGTCP core. Red Team Ltd recommends keeping `wgtcp` out of every regional `transport_order` until all four issues are resolved.

Transport fingerprintability. The upgrade request, the Noise framing, endpoint and decoy selection, and the WGTCP records were captured (T14, T16) and reviewed. The cleartext half of the RedSocket opening carries no device identifier, which the core's own test `upgrade_request_leaks_no_identifier` guards, and the `cid` travels only inside Noise. The design supports decoy host pools, varied paths, CDN fronts and a raced fallback tier, and the 8-second handshake budget prevents a censor from stalling a dial indefinitely. The opening itself, however, is a fixed cleartext dialogue that one stateless rule matches. A default `Host` impersonating a major identity provider makes it more conspicuous, and per-install choices link a user's sessions (RVPN-01-410). The development team's answer for the HTTP carrier is the Proteus transport, assessed in WP2, which replaces the HTTP front while the HTTP carrier is retained for legacy clients. The iOS client at 1b196be still ships only the HTTP carrier, so the finding remains open for this build, and so does its WGTCP part.

C wrapper and FFI boundary. Both hand-written headers were compared with the archives' exports, and the Rust FFI modules and Swift wrappers were reviewed. This area held up best. The headers match the exports exactly (33 of 33 and 6 of 6, T04, unchanged in T25), with matching argument and return types. Every string-returning entry point runs inside `catch_unwind` and returns an error string instead of unwinding into Swift. Strings allocated by Rust are released only through the matching free functions. The Swift wrappers copy them before release, pass Swift strings through `withCString` for the duration of a call only, and run the blocking start calls on background threads. The setter functions that run outside `catch_unwind` would abort rather than unwind on a panic, an acceptable outcome given that the analysed code cannot reach those panics. The stale bridge header was removed in 0af77e2; the Android entry points in the iOS core remain (RVPN-01-416).

Prebuilt-binary provenance and reproducibility. All six archives were examined by load commands, symbols, embedded strings and rebuilds (T01 to T08). At bd9bbc6, RedSocketCore could not be reproduced: it had been

built from a different configuration and differed in five functions (RVPN-01-415). The maintainers responded during the engagement. Commit 10433da replaced all four core slices with archives built from the published source in a neutral directory and recorded the source revision in the README. Red Team Ltd then reproduced RedSocketCore bit for bit and WgtcpCore function for function, and found no build-host path in any slice (T25 to T27; RVPN-01-415, RVPN-01-417). The rest of the provenance chain remains open. The bridge still comes from an unpinned upstream, local SwiftPM binary targets carry no checksum, and no attestation or SBOM accompanies the binaries. These map onto NIST SP 800-218 practices PS.2 and PS.3 and onto SLSA v1.0 Build levels 1 to 3, none of which is currently met. To the extent the app is available in the EU, the Cyber Resilience Act's vulnerability-reporting duties under Article 14 apply from 11 September 2026, and its remaining obligations, including SBOMs, from 11 December 2027. CI-built, attested artefacts with CycloneDX 1.7 or SPDX 3.0.1 SBOMs would serve both.

SwiftPM and vendored dependencies. Package manifests, resolution files, vendored trees and the three Cargo lock files were reviewed and tested (T09 to T13, T21). All Swift packages are local, `Package.resolved` carries no pins, and SwiftPM reports no external dependencies. No analytics, advertising, crash-reporting or WebView components exist, and no URL schemes, universal links or background modes widen the attack surface. `cargo audit` reported no advisories for any of the three Rust lock files, and all Rust sources are vendored with checksums. Since 0af77e2 the unlinked libyaml archive is gone. On the debit side, the vendored libyaml source tree is still present and unused, one locked crate is a yanked release, and Android-only crates are linked into the iOS binaries. The bridge archive is still device-only, so the simulator builds of the remediated tree (T28, T29) relied on a converted copy outside the repository (RVPN-01-416).

Privacy statements. The in-app privacy policy states that device identifiers, the mobile carrier and third-party analytics "may" be collected. The code collects none of these, so the policy should be narrowed rather than the code widened. The Terms' statement that no record of visited sites is kept is consistent with the client, subject to the rules-upload observation (RVPN-01-409).

Standards. Against MASVS v2.1.0 at 1b196be (Appendix C), the client fails NETWORK-1, NETWORK-2 and PLATFORM-1. It partially meets STORAGE-1, STORAGE-2, CRYPTO-1, CRYPTO-2, AUTH-1, CODE-3, CODE-4 and all four PRIVACY controls, and passes PLATFORM-2, PLATFORM-3 and CODE-1; the resilience controls were out of scope by design. In the OWASP Mobile Top 10 (2024), the remaining findings concentrate in M5 (Insecure Communication), M6 (Inadequate Privacy Controls) and M2 (Inadequate Supply Chain Security), with the WireGuard key in backed-up defaults under M9 (Insecure Data Storage). The project builds in Swift 5 language mode on a Swift 6.4 toolchain, and an upgrade to Swift 6 mode is warranted given the data races noted in RVPN-01-416. The RedSocket upgrade does not conform to RFC 6455, and both SOCKS servers deviate from RFC 1928 method negotiation; both deviations should be corrected rather than documented.

Verdict. After remediation the client is markedly stronger: IPv6 can no longer bypass the tunnel, logs no longer expose sessions, keychain items are bound to the device, and the launch-time cashier calls wait until the tunnel state is known. It is not yet fit for users who face the full threat model. The tunnel still fails open, the carriers shipped in this build are still fingerprintable, and the WGTCP path must stay disabled. The remaining work is concrete. The carrier revision, which the maintainers address with Proteus, is the one item that needs coordination across work packages.

Prioritised recommendations.

1. Decide on fail-closed operation, through `includeAllNetworks` with on-demand rules or at least a user-selectable kill switch, and add an upstream-aware health check (RVPN-01-402).
2. Keep WGTCP disabled until resolution, readiness, profile signing and carrier framing are fixed (RVPN-01-414, RVPN-01-413, RVPN-01-407, RVPN-01-410).
3. Bring the Proteus transport to iOS and retire the HTTP carrier as soon as legacy clients allow; until then, remove the per-install stable choices (RVPN-01-410).
4. Route the configuration fetch and WGTCP provisioning through the tunnel or a fronted base, and replace the shared API key (RVPN-01-406).
5. Authenticate the loopback proxy, pass the bound port to internal clients, and sign the blocklist (RVPN-01-411, RVPN-01-412).
6. Move the WireGuard key into a keychain item the extension can read, and exclude the shared defaults from backup (RVPN-01-404).

7. Add freshness to the signed configuration and authenticate state-changing notes (RVPN-01-408, RVPN-01-409).
8. Extend the reproducible builds to the bridge, ship it with a simulator slice, and move all native builds into CI with SLSA provenance and SBOMs (RVPN-01-415, RVPN-01-416).
9. Remove or complete the relay, including the “Help Improve” toggle (RVPN-01-405).

Red Team Ltd would like to thank the RostamVPN maintainers for publishing the client and its cores under an open licence, for the detailed comments that made the code’s intent easy to follow, and for a prompt remediation pass whose small, focused commits made verification straightforward.

Appendix A: Dynamic Test Log

All tests ran on 2026-10-14 (US Pacific time) on the assessment host described under Tools. Shell variables in the commands stand for host paths and withheld values: `$IOS` is the iOS working tree at bd9bbc6 and `$ANDROID` the Android working tree at f60abd3; `$ARCHIVES` lists the six archives named in Scope and `$A` is one of them; `$MEMBERS` is a scratch directory holding the extracted members of one core; `$SHIPPED_OBJ` and `$REBUILT_OBJ` are the shipped and rebuilt object of the same crate; `$FLEET_CONSTANT` is the constant at `BrandRuntime.swift:154–155` (not reproduced here); `$PATTERN` and `$PRIVATE_OUT` are explained in the tests that use them. In T27 to T31, `$IOS` is the working tree at 1b196be, `$BASE` the copy of bd9bbc6 at `/tmp/rvpn01/wp4/ios-copy`, and `$FIX` the development team’s scratch directory `/tmp/rvpn01/fix-wp4`, which holds their helper scripts and a synchronised copy of 1b196be. Scratch paths are under `/tmp/rvpn01/wp4`, except where T28 and T29 use `$FIX`. The three Android subtrees were copied to `/tmp/rvpn01/wp4/src-copy` at 19:15 local time (verified identical with `diff -rq` at that time) so that no build touched the Android repository. Every process started was recorded and stopped by PID or exited on its own; no listener remained in 127.0.0.1:20400 to 20499 at the end (`lsof` check). Outputs are trimmed; key material generated per run is elided.

T01: Archive inventory

Command: `stat`, `shasum -a 256` and `lipo -info` over the six archives under `$IOS/LocalPackages`. Result: pass. All six are arm64-only; sizes and hashes are as listed in Scope; the bridge archive is dated 2026-09-22, libyaml 2026-09-19, and both cores 2026-10-14.

T02: Load commands

Command: `for A in $ARCHIVES; do otool -l "$A" | grep -A4 -E 'LC_BUILD_VERSION|LC_VERSION_MIN'; done` over the six archives, then `ar -x` of the WgtcpCore device slice and a per-member `minos` check. Result: finding (RVPN-01-416). RedSocketCore has 469 members; Rust objects carry `LC_VERSION_MIN_IPHONEOS 10.0` on the device slice and `minos 14.0` on the simulator slice. WgtcpCore has 488 members, 24 of which (ring: `aes_nohw.o`, `curve25519.o`, `montgomery.o`, `p256.o`, `poly1305.o`, the `*-ios64.o` assembly objects and others) carry `platform 2 minos 27.0 sdk 27.0`. The bridge archive’s 169 object files target `minos 15.0`, built with SDK 26.1.

T03: Embedded path strings

Command: `cat "$MEMBERS"/*.o | strings -a | grep -o -E "$PATH_RE" | sort | uniq -c` on the extracted members of each core, where `PATH_RE` matches registry, toolchain and workspace path forms. Result: finding (RVPN-01-415, RVPN-01-417). RedSocketCore: 300 registry paths of the form `x/.cargo/registry/src/index.crates.io-1949cf8c6b5b557f/` followed by a crate directory, toolchain paths `x/.rustup/toolchains/stable-aarch64-apple-darwin`, crate sources as relative `src/*.rs` names, and no vendored paths. WgtcpCore: 433 paths under the build host’s remapped workspace, `.../wgtcp-android/vendor/*` and `.../vendor/redsocket_wgtcp/src`, plus 101 toolchain paths under the build host’s account (values withheld). The standard library’s own CI paths appear in both.

T04: FFI exports versus headers

Command: `nm -gU "$A" | awk '{print $3}' | grep -E '^_(rs_|wgtcp_|Java_|JNI_)'` for each core archive, compared with the function names in the headers. Result: pass with note. 33 `rs_*` exports equal the 33 declarations;

6 `wgtcp_*` exports equal the 6 declarations; device and simulator export lists are identical; RedSocketCore additionally exports 34 `Java_com_rostamvpn_android_*/JNI_OnLoad` symbols.

T05: Bridge archive exports and members

Command: `nm -gU libhev-socks5-tunnel.a`, `ar -t`, and `strings -a`. Result: finding (RVPN-01-416). Exports are `hev_socks5_tunnel_{init,run,stop,fini,main,main_from_file,main_from_str,quit,stats,update_session}`. The header `hev_socks5_tunnel.h` declares `hev_socks5_tunnel_get_stats` and `hev_socks5_tunnel_set_tunnel_fd`, which are absent. Members include `hev-mapped-dns.o` and the lwIP objects. libyaml `0.2.5` is embedded, with 58 `yaml_*` exports. No version string identifies the bridge.

T06: RedSocketCore reproduction

Environment: copy of `redsocket-http-core` without `.cargo` or `vendor`, so that sources resolve from the local registry cache. Command:

```
RUSTFLAGS="--remap-path-prefix=$HOME=x" CARGO_TARGET_DIR=/tmp/rvpn01/wp4/target/repro-rs \
  cargo build --release --locked --offline --lib --target aarch64-apple-ios # and aarch64-apple-ios-sim
```

Result: finding (RVPN-01-415). The device build finished in 14.76 s and the simulator build in 13.46 s. Rebuilt sizes are 33,997,376 and 33,969,288 bytes, against 33,996,992 and 33,968,904 shipped. Member names (crate hashes) are identical; 467 of 469 members are byte-identical; the differences are `__.SYMDEF` and `redsocket_core.rcgu.o` ↪ `redsocket_core.daede021ed215a58-cgu.0.rcgu.o` (4,721,672 bytes shipped, 4,722,136 rebuilt). Defined function names differ only by `socks5::Upstream::active_label`.

T07: WgtcpCore rebuild with the published configuration

Command: `CARGO_TARGET_DIR=/tmp/rvpn01/wp4/target/build-wg cargo build --release --locked --offline --lib` ↪ `--target aarch64-apple-ios` (and `-sim`) in the copied `wgtcp-android`. A first attempt failed with `failed to open` ↪ `file .../vendor/cc/src/target/llvm.rs`. The cause was the testing team's copy step, an `rsync --exclude` ↪ `target` rule that also removed the `cc` crate's legitimate `src/target` directory; after a corrected copy was verified identical, the build passed. Result: 488 members, 424 byte-identical, 61 differing and 3 with different names. The differences follow the build path.

T08: WgtcpCore rebuild with the shipped path prefix; per-function comparison

Command: as T07, but with `RUSTFLAGS` and `CFLAGS` remapping the scratch path to the prefix observed in the shipped archive (`SHIPPED_PREFIX`, value withheld; see RVPN-01-417). The per-function comparison then ran `python3 -I fnsizes.py "$SHIPPED_OBJ" "$REBUILT_OBJ"` for each first-party object, comparing `nm --no-llvm-bc -n` ↪ `-U` text symbols and their sizes. Result: WgtcpCore matches the source; RedSocketCore does not.

```
common=485 identical=484 differing=1 (__.SYMDEF)
redsocket_wgtcp object: sections equal; functions shipped=259 rebuilt=259; differing 0
wgtcp_core object: sections equal; functions shipped=249 rebuilt=249; differing 0
redsocket_core object: functions shipped=708 rebuilt=707; differing 5 (see RVPN-01-415)
```

T09: Host tests of redsocket-http-core

Command: `CARGO_TARGET_DIR=/tmp/rvpn01/wp4/target/test-rs cargo test --locked --offline -- --test-threads` ↪ `=1`. Before the first run a placeholder containing `{}` was created in the scratch tree at `android/gen/android` ↪ `/app/src/main/assets/default-config.json`, a generated Android asset outside the permitted scope. The first run still did not compile: `couldn't read src/../../brands/configs/rostam.json` (another `include_str!` to a file outside the scope). A second placeholder was created at that path for the second run. Result: 123 passed, 13 failed, 6 ignored. All 13 failures are in `config.rs` tests that load the placeholder files (for example `shipped_config_yields_a_usable_endpoint_list` and `the_shipped_config_enables_mux`) and are not attributable to the code under test. The six ignored tests contact live hosts and were not run.

T10: Host tests of wgtcp-android

Command: `cargo test --locked --offline` (after the T07 copy correction; the first attempt failed for the same copy reason). Result: pass, 3 passed.

T11: Host tests of redsocket_wgtcp

Command: `cargo test --locked --offline`. Result: pass, 82 passed (library) and 2 passed (second test target), 0 failed.

T12: cargo audit

Command: `cargo audit --no-fetch --stale -f Cargo.lock` in each copied crate. Result: 1,329 advisories loaded. redsocket-http-core: 171 dependencies, no vulnerabilities, no warnings. wgtcp-android: 196 dependencies, no vulnerabilities, one warning (`chacha20 0.10.1` yanked). redsocket_wgtcp: 172 dependencies, no vulnerabilities, the same warning.

T13: cargo tree

Command: `cargo tree --offline --locked -i chacha20@0.10.1 -e normal`; `cargo tree ... --target aarch64-apple-ios -i jni` and `-i android_logger`. Result: `chacha20 0.10.1` comes from `rand 0.10.2`, which comes from `redsocket-wgtcp`. `jni` and `android_logger` are normal dependencies on the iOS target for both cores.

T14: RedSocket wire capture and SOCKS negotiation

Setup: the harness `wp4_rs.rs`, built as an example of the copied crate, starts a fake exit (snow `Noise_NK_25519_ChaChaPoly_BLAKE2s` responder with a per-run key) on 127.0.0.1:20401. It then calls `rs_set_client_id`, `rs_set_premium(0)`, `rs_set_region("")`, `rs_set_rate_limit_kbps(0)`, `rs_set_cdn_allowed(0)`, `rs_load_config`, `rs_set_app("rostam")` and `rs_start(20402)`, in the provider's order. Command: `GAI_LOG=... DYLD_INSERT_LIBRARIES=/tmp/rvpn01/wp4/harness/libgai.dylib ./wp4_rs wire`. Result: findings (RVPN-01-410, RVPN-01-411); positive result for name handling (no resolver call).

```
[exit#1] cleartext request (97 bytes): GET /noise HTTP/1.1\r\nHost: login.microsoftonline.com\r\nUpgrade:
    ↪ websocket\r\nConnection: Upgrade\r\n\r\n
[exit#1] first client bytes after 101: length prefix 0030 then 48 bytes
[exit#1] sent msg2 (48 bytes)
[exit#1] transport frame 1 (56 bytes ciphertext) plaintext: META||||0123456789abcdef0123456789abcdef
[exit#1] client closed after 1 transport frame(s)
rs_start(20402) -> "ok:http:127.0.0.1:20401" in 3 ms; rs_is_running=true rs_socks_port=20402
SOCKS5 greeting offering ONLY method 0x02 -> server selected 0500
[exit#2] transport frame 2 (33 bytes ciphertext) plaintext: \x05\x03\x0cdest.example\x00P
SOCKS5 CONNECT dest.example:80 (by name) -> reply 05000001000000000000 after 3 ms
--- getaddrinfo calls: (none recorded)
```

T15: SOCKS port squatting and blocklist substitution

Setup: `python3 -I squatter.py 20410 60` (a SOCKS5 responder that serves a forged list), then `./wp4_rs squat`. The configuration adds `"blocklist":{"enabled":true,"url":"http://lists.example.test/v1.json"}`. The squatter was stopped by PID. Result: finding (RVPN-01-412).

```
rs_start(20410) -> "ok:http:127.0.0.1:20401" in 0 ms; rs_is_running=true rs_socks_port=57696
[squatter] conn from ('127.0.0.1', 57709): greeting ver=5 methods=00
[squatter] CONNECT cmd=1 to lists.example.test:80
[squatter] served forged blocklist (86 bytes)
rs_get_block_stats (blocking still off) -> {"armed":false,"blocked_total":0,"domains":1,"enabled":false,"
    ↪ list_version":4242,...}
CONNECT forged-block.example.test:443 with blocking OFF -> reply 05000001000000000000
CONNECT forged-block.example.test:443 with blocking ON -> reply 05020001000000000000
rs_get_block_stats -> {"armed":true,"blocked_total":1,"by_category":{"wp4-forged":1},...}
```

The OS-assigned port 57696 lies outside the assigned block because the core chooses it itself; that fallback is the behaviour under test.

T16: WGTCP readiness, carrier prefix and name resolution

Setup: `python3 -I silent.py 20420 60|75` (accepts and never answers); harness `wp4_wg.rs` calls `wgtcp_generate_keypair` twice and starts `wgtcp_start(config, 20421)` with endpoint 127.0.0.1:20420 and address 100.64.0.2/24, under the `getaddrinfo` interposer. One initial run was followed by two extended runs with fresh keys, and each listener was stopped by PID. Result: findings (RVPN-01-413, RVPN-01-414, RVPN-01-410).

```
wgtcp_start -> "ok" after 210 ms; wgtcp_status="connected"; wgtcp_is_running=1
CONNECT by name wp4-dns-probe.invalid:80 -> reply 05040001000000000000 after 0 ms
[gai] getaddrinfo(node=wp4-dns-probe.invalid, service=(null))
CONNECT by IPv4 192.0.2.10:80 -> reply 05000001000000000000 after 0 ms
SOCKS reply 05000001000000000000 then request sent; bytes back within 10 s: Err(WouldBlock) after 10001
    ↪ ms
25 s later (two 10 s health-check periods): wgtcp_status="connected"; wgtcp_is_running=1; local SOCKS
    ↪ port accepting=true
[silent] run 1: 156 bytes at +0 s, 312 at +5 s, 468 at +10 s, 512 at +15 s; first 16: 0000009
    ↪ c0000b6250100000001000000
[silent] run 2: 156, 312, 468, 512 bytes on the same schedule; first 16: 0000009c0000b6250100000001000000
```

T17: RedSocket upstream blackhole versus provider health signals

Command: `./wp4_rs blackhole`, which establishes a session, then shuts down the fake exit and samples the provider's two health signals every 10 s. Result: finding (RVPN-01-402).

```
rs_start(20402) -> "ok:http:127.0.0.1:20401" in 1 ms
CONNECT dest.example:80 while the exit is up -> reply 05000001000000000000 after 1 ms
fake exit stopped (listener closed); exit now unreachable
health signals after 10 s: rs_is_running=true local SOCKS port accepting=true | CONNECT dest.example:80
    ↪ -> reply 05010001000000000000
health signals after 20 s: rs_is_running=true local SOCKS port accepting=true | CONNECT dest.example:80
    ↪ -> reply 05010001000000000000
rs_stop done; rs_is_running=false
```

T18: Configuration seal test vector

Command: the vector and constant were extracted from `cfgseal.rs` (test `opens_a_config_sealed_by_the_real_publisher`); then `./unseal-config sealed-test-vector.b64 "$FLEET_CONSTANT"`, and the same without the constant. Result: finding (RVPN-01-408, RVPN-01-417). With the constant the vector opens: exit 0, 92 bytes, keys `transport` and `version` (`"rt-test"`), no time fields, and one public IPv4 endpoint (value withheld). Without the constant: `cfgseal-v2-dualkey: config did not decrypt with either key`, exit 1.

T19: Static presence checks

Command: `grep -rn -E "$PATTERN" --include='*.swift' --include='*.plist' --include='*.entitlements' --include='*.xcconfig' --include='*.pbxproj' "$IOS"`, with one pattern per control. Result: findings (multiple). Each of the following returned zero matches: IPv6 settings, `includeAllNetworks`/`excludeLocalNetworks` /`enforceRoutes`, on-demand rules, `sleep`/`wake` overrides, path monitoring, `ThisDeviceOnly`, `kSecAttrAccessGroup`, backup exclusion, complete file protection, trust evaluation or pinning, ATS keys, `Tun2SocksWrapper.shared.stop`, `Tun2SocksWrapper.shared.running`, `showLog = true`, `aps-environment`, `BGTaskSchedulerPermittedIdentifiers`, `UIBackgroundModes`, `UIPasteboard`, WebViews, ATT/IDFA/IDFV, and jailbreak or debugger checks. `%{public}` had one match (`RedSocketPacketTunnelProvider.swift:45`). `healthTimer?.cancel()` has one match, at `:697` (`startHealthMonitor`), not in `stopTunnel`. `userOptIn` is assigned only inside its own setter, and `"config-updated"` occurs only in the app (`RostamVPNApp.swift:203`).

T20: Redaction scan

Command: `python3 -I redact_scan.py "$IOS" "$PRIVATE_OUT"` (public IPv4 outside RFC 5737, mailboxes, CDN distribution names, home-directory paths, OAuth client identifiers, hex values of 40 or more digits, over text and binaries), plus `strings` over the archives for host names. Result: finding (RVPN-01-417); verification of the Fix Note (RVPN-01-418). Matches were classified privately and are reported by class only. Archive host names were limited to the compiled default decoy host, `index.crates.io` paths, `openssl.org` attribution and YAML tags.

T21: SwiftPM resolution review

Command: `swift package show-dependencies` and `swift package describe --type json` in copies of both local packages. Result: pass. Both report "No external dependencies found"; `rostam-apple` has two binary targets plus `RostamKit` and its tests, and `hev-socks5-tunnel-apple` has two library targets. Both `Package.resolved` files carry `"pins": []`.

T22: Property-list lint

Command: `plutil -lint` on both entitlements files, both Info.plist files, the export options and both xcframework `Info.plist` files. Result: pass, all seven files `OK`.

T23: Simulator compile

Setup: a copy of `$IOS` without `.git` at `/tmp/rvpn01/wp4/ios-copy`, where `Scripts/select-brand.sh rostam` was run. Command:

```
xcodebuild -workspace rostamvpn.xcworkspace -scheme rostamvpn -configuration Release \
  -sdk iphonesimulator -destination 'generic/platform=iOS␣Simulator' \
  -derivedDataPath /tmp/rvpn01/wp4/build/dd CODE_SIGNING_ALLOWED=NO CODE_SIGNING_REQUIRED=NO build
```

Result: fail, exit 65 after 20 seconds (RVPN-01-416). RostamKit, the bridge wrapper and the extension compiled; the extension link failed for arm64 with `ld: building for 'iOS-simulator', but linking in object ↪ file (...libhev-socks5-tunnel.a[2](hev-rbtree.o))built for 'iOS'`, and for x86_64 with `found architecture ↪ 'arm64', required architecture 'x86_64'`. The app target was therefore not compiled. A second attempt (arm64 only, `/tmp/rvpn01/wp4/build/dd2`) followed a `vtool -set-build-version` step on a scratch copy of the bridge archive. That step re-stamped no objects, so the attempt failed identically (exit 65). Simulator work then ended at the engagement lead's instruction. No device was used.

T24: Remediation check, debug fixture (dcb2d05)

Command: a Python scan of every IPv4 literal in `rostamvpn/Tabs/IPGuardian.swift` at b560865, classified against RFC 5737. Result: pass; fix verified (RVPN-01-417). Lines 62, 64 and 80 hold only documentation addresses.

T25: Remediation check, path strings in the cores (b560865)

Command: for each of the four replaced slices, `strings -a "$A" | grep -c -F "$(id -un)"`, `strings -a "$A" | ↪ grep -c -E "$HOME_RE"` (`HOME_RE` matches the macOS home root and `/home/`), a count of the workspace-name pattern, and a histogram of embedded path prefixes; `nm -gU` for exports; `otool -l` for load commands. Result: pass; fix verified (RVPN-01-417). All three counts are 0 in every slice. Source prefixes are `/build/rostamvpn ↪ -android/redsocket-http-core/vendor/...` and `/build/rostamvpn-android/wgtcp-android/vendor/...`, and the toolchain prefix is `/rustup/toolchains/stable-aarch64-apple-darwin`. Exports are unchanged (33 `rs_*` and 34 JNI symbols; 6 `wgtcp_*`), and the 24 ring objects per WgtcpCore slice still carry `minos 27.0` (RVPN-01-416).

T26: Remediation check, reproduction of the cores (b560865)

Environment: the three Android subtrees were copied again at 22:35 local time from the working tree that follows 621ba38. `diff -rq` against the f60abd3 copy showed no difference apart from the harness files added in T14 and T16. Command, for both crates and both targets:

```
SRC=/private/tmp/rvpn01/wp4/src-copy-621ba38
RUSTFLAGS="--remap-path-prefix=$SRC=/build/rostamvpn-android␣--remap-path-prefix=$HOME/.rustup=/rustup" \
CFLAGS="-ffile-prefix-map=$SRC=/build/rostamvpn-android␣-ffile-prefix-map=$HOME/.rustup=/rustup" \
  cargo build --release --locked --offline --lib --target aarch64-apple-ios # and aarch64-apple-ios-sim
```

Result: pass; fix verified (RVPN-01-415).

```
RedSocketCore ios-arm64: shipped 33,991,408 bytes 0866269f4d72e46f = rebuilt 33,991,408 bytes 0866269
    ↪ f4d72e46f
RedSocketCore simulator: shipped 33,963,320 bytes ef2f1c057af152f7 = rebuilt 33,963,320 bytes
    ↪ ef2f1c057af152f7
WgtcpCore ios-arm64: 484 dependency members identical; first-party functions 259/259 and 249/249 equal in
    ↪  size
WgtcpCore simulator: 484 dependency members identical; first-party functions 259/259 and 249/249 equal in
    ↪  size
RedSocketCore bd9bbc6 vs b560865 crate object: 708 vs 707 functions, the five functions of RVPN-01-415
    ↪ differ
```

The three first-party WgtcpCore members carry different file names in the rebuild, because a path dependency's crate hash depends on its absolute location.

T27: Remediation check, changes from bd9bbc6 to 1b196be

Command: `diff -rq --exclude .git --exclude .build --exclude audit-results.txt --exclude 'PENTEST-REPORT
↪ .*' --exclude Brand.xcconfig --exclude Brand --exclude libhev-socks5-tunnel.a "$IOS" "$BASE"`, then `diff
↪ -u` and a full read of every changed file, `shasum -a 256` over the five archives, and the T19 patterns again. Result: pass with open items, recorded in the Fix Note of each issue. Condensed output:

```
added: RostamKit/ControlPlane.swift, RostamKit/TunnelNetwork.swift, ControlPlaneTests.swift,
    ↪ GuardianMessageTests.swift, TunnelNetworkTests.swift
removed: CHevSocks5Tunnel/hev_socks5_tunnel.h, CHevSocks5Tunnel/out/libyaml.a
changed: RedSocketPacketTunnelProvider.swift, RostamVPNApp.swift, CashierApi.swift, KeychainService.swift
    ↪ , Store.swift,
       IPGuardStore.swift, IPGuardian.swift, Localizable.xcstrings, Guardian.swift, PeerActivation.swift,
       HevSocks5Tunnel.swift, README.md and the four core archives
archives: RedSocketCore 0866269f4d72e46f / ef2f1c057af152f7, WgtcpCore 8c48db619ee9582e / 1
    ↪ f1c5ab800ee70ad (as in T25, T26);
       bridge 9d2bdb3d2ea842a0 (unchanged)
T19 patterns now matching: NEIPv6Settings|NEIPv6Route 2, matchDomains 2, ThisDeviceOnly 1,
    ↪ isExcludedFromBackup 1,
       Tun2SocksWrapper.shared.stop 1, healthTimer?.cancel() in stopTunnel
still no match: includeAllNetworks|excludeLocalNetworks|enforceRoutes, on-demand rules, sleep/wake
    ↪ overrides,
       NWPathMonitor, kSecAttrAccessGroup, trust evaluation or pinning
%{public}: 0 matches; %{private}: 3 (extLog, vpnLog, payLog)
```

T28: Remediation check, development team's build and test script

Command: `cd "$FIX" && ./tools/verify.sh wp4-reverify`. The script synchronises `$IOS` into `$FIX/ios` (without `.
↪ git`, the report, the brand outputs and the bridge archive), builds the `rostamvpn` scheme for the simulator (Release, arm64, signing disabled) into `$FIX/dd`, compares the diagnostics with the development team's baseline, and runs `xcodebuild test -scheme rostam-apple-Package` on an iPhone 17 Pro Max simulator with iOS 26.5. Result: pass.

```
build exit=0; ** BUILD SUCCEEDED **
--- gone vs baseline: hev-main.h: warning: umbrella header for module 'CHevSocks5Tunnel' does not include
    ↪  header 'hev_socks5_tunnel.h'
test exit=0
Test Suite 'ControlPlaneTests' passed: Executed 1 test, with 0 failures
Test Suite 'EntitlementMigrationTests' passed: Executed 7 tests, with 0 failures
Test Suite 'GuardianMessageTests' passed: Executed 1 test, with 0 failures
Test Suite 'TunnelNetworkTests' passed: Executed 1 test, with 0 failures
Executed 10 tests, with 0 failures (0 unexpected) in 0.026 (0.031) seconds
```

The build was incremental: its log contains no compile or link action, because the same sources had already been built in `$FIX/dd`. T29 supplies a clean build.

T29: Remediation check, clean simulator build

Setup: `$FIX/ios` as synchronised in T28. Its bridge archive had been converted to the simulator platform with the development team's `to_sim.py`, because the archive in the repository is still device-only. Command:

```
cd "$FIX/ios" && xcodebuild -workspace rostamvpn.xcworkspace -scheme rostamvpn -configuration Release \
  -sdk iphonesimulator -destination 'generic/platform=iOS␣Simulator' -derivedDataPath /tmp/rvpn01/wp4/dd-
      ↪ clean \
  ARCHS=arm64 ONLY_ACTIVE_ARCH=YES DEBUG_INFORMATION_FORMAT=dwarf COMPILER_INDEX_STORE_ENABLE=NO \
  CODE_SIGNING_ALLOWED=NO CODE_SIGNING_REQUIRED=NO clean build
```

Result: pass with note (RVPN-01-416). `** CLEAN SUCCEEDED **` and `** BUILD SUCCEEDED **` after 20 seconds. Swift compilation ran for RostamKit, the bridge wrapper, the extension and the app, followed by five link steps (`CHevSocks5Tunnel.o`, `HevSocks5TunnelWrapper.o`, `RostamKit.o`, `RedSocketTunnel`, `RostamVPN`). The linker printed 42 warnings that ring objects were "built for newer 'iOS-simulator' version (27.0) than being linked (15.6)". The repository's own bridge archive is unchanged since T23 (same SHA-256), where the extension link failed with it. The derived data was deleted afterwards.

T30: Remediation check, loopback discovery of the SOCKS port

Setup: since bf88011 the extension picks its preferred port at random between 20000 and 49151. A stand-in for the core, `python3 -I scripts/standin.py 20`, listened on a random port of the assigned block and answered a SOCKS5 greeting with `05 00`, as the core did in T14. Command: `python3 -I scripts/scanner.py`, which connects to each port from 20400 to 20499 and sends `05 01 00`. The stand-in was then stopped by PID. Result: finding confirmed (RVPN-01-411).

```
listening on 20421
scanned 100 loopback ports in 4.5 ms; SOCKS5 no-auth servers found: [20421]
```

At this rate, the extension's whole range of 29,152 ports takes about 1.3 seconds. That figure is an extrapolation; only the assigned block was scanned.

T31: Remediation check, backup exclusion of the Guardian history

Command: `swift -module-cache-path /tmp/rvpn01/wp4/swift-cache "$FIX/tools/backup_flag_probe.swift" /tmp/
↪ rvpn01/wp4/probe-dir`. The probe repeats the write sequence of `IPGuardStore.save` (an atomic write, then `isExcludedFromBackup = true`) on the host's APFS volume, rewrites the file once without setting the flag, and reads the flag after each step. Result: pass (RVPN-01-404).

```
after save+flag: true
after a bare atomic rewrite: true
after save+flag again: true
```

The flag survived the bare rewrite on the host, and the implementation re-applies it on every save in any case. Backup behaviour on a device was not tested.

Appendix B: Dependency Inventory and Advisories

Swift and native layer.

Component	Version	Source	Notes
RostamKit	first-party	`LocalPackages/rostam-` `↪ apple`	Wraps both cores; tools version 5.9

Component	Version	Source	Notes
HevSocks5TunnelWrapper	first-party	`LocalPackages/hev-socks5-tunnel-apple`	Swift bridge and C headers
hev-socks5-tunnel	not determinable	prebuilt `libhev-socks5-tunnel.a`	Built from unpinned upstream HEAD; device slice only
lwIP	not determinable	inside the bridge archive	No version marker
libyaml	0.2.5	inside the bridge archive	Separate `libyaml.a` removed in 0af77e2; vendored source still unused; YAML input is generated by the app

SwiftPM: no remote packages; `Package.resolved` pins are empty in both workspace locations (T21).

Rust cores (cargo-audit 0.22.2, advisory database edb58f59, 1,329 advisories).

Lock file	Crates	Advisories	Warnings
`redsocket-http-core/Cargo.lock`	171	none	none
`wgtcp-android/Cargo.lock`	196	none	`chacha20` 0.10.1 yanked
`vendor/redsocket_wgtcp/Cargo.lock`	172	none	`chacha20` 0.10.1 yanked

Security-relevant crates.

Crate	Version	Used by	Role
snow	0.9.6	RedSocket	Noise NK
yamux	0.13.10	RedSocket	MUX2 and relay
ed25519-dalek	2.2.0	RedSocket	Configuration signature
chacha20poly1305	0.10.1	both	AEAD for the seal, Noise and WireGuard
curve25519-dalek	4.1.3	both	X25519
x25519-dalek	2.0.1	WGTCP	WireGuard keys
blake2	0.10.6	both	Noise hash, seal KDF
boringtun	0.7.1	WGTCP	WireGuard state machine
ring	0.17.14	WGTCP	Crypto via boringtun
smoltcp	0.13.1	WGTCP	Inner TCP/IP stack
tokio	1.53.1	both	Runtime
serde_json	1.0.151	both	Parsing
rand	0.8.7, 0.9.5 / 0.10.2	RedSocket / WGTCP	Randomness
getrandom	0.2.17, 0.3.4 / 0.4.3	RedSocket / WGTCP	OS CSPRNG
jni, android_logger	0.21.1, 0.14.1	both	Android only, linked on iOS
log	0.4.33	both	`release_max_level_warn`; no logger on iOS

All Rust dependencies are vendored with checksums in the Android tree. At bd9bbc6 the shipped RedSocket-Core had nonetheless been compiled from the registry cache; since 10433da, the amended form of b560865, both cores are built from the vendored tree and reproduce from it (RVPN-01-415, T26, T27). No SBOM is published. Red Team Ltd recommends CycloneDX 1.7 or SPDX 3.0.1, generated in CI from the lock files and the native components.

Appendix C: Standards Conformance Checklist (MASVS v2.1.0)

Control	Result	Note
MASVS-STORAGE-1	Partial	Keychain items device-bound since bc011b9; WireGuard key and `cid` still in backed-up shared defaults (RVPN-01-404)
MASVS-STORAGE-2	Partial	Logs private and shared log file removed since 3ba2f29; shared defaults still backed up (RVPN-01-403, RVPN-01-404)
MASVS-CRYPTO-1	Partial	Noise NK, WireGuard and Ed25519 sound; seal's symmetric layer public (RVPN-01-408); zero-key export removed in 7b72411
MASVS-CRYPTO-2	Partial	Keys from the OS CSPRNG; WireGuard private key copied out of the keychain (RVPN-01-404)
MASVS-AUTH-1	Partial	Self-asserted `cid`, shared API key, unauthenticated notes (RVPN-01-406, RVPN-01-409)
MASVS-AUTH-2	Not assessed	No local authentication feature
MASVS-AUTH-3	Not assessed	No step-up operations in a VPN client
MASVS-NETWORK-1	Fail	IPv6 claimed since 1131ebf; fail-open, carrier signatures, WGTCP readiness and resolution remain (RVPN-01-402, -410, -413, -414)
MASVS-NETWORK-2	Fail	No pinning; unsigned WGTCP profile (RVPN-01-407)
MASVS-PLATFORM-1	Fail	Random port since bf88011, but the loopback proxy still serves every local process; port misdirection in the core (RVPN-01-411, RVPN-01-412)
MASVS-PLATFORM-2	Pass	No WebViews
MASVS-PLATFORM-3	Pass	No sensitive data rendered in the `classic` composition
MASVS-CODE-1	Pass	Deployment target iOS 15.6; current SDK
MASVS-CODE-2	Not assessed	App Store distribution; no in-app update mechanism
MASVS-CODE-3	Partial	No advisories; yanked crate and remaining build defects (RVPN-01-416); cores reproducible, bridge provenance open (RVPN-01-415)
MASVS-CODE-4	Partial	Signed configuration verified; no freshness; future rules dates clamped, notes and blocklist unauthenticated (RVPN-01-408, -409, -412)

Control	Result	Note
MASVS-RESILIENCE-1	Not assessed	Not expected for an open-source circumvention client
MASVS-RESILIENCE-2	Not assessed	App Store signature only, by design
MASVS-RESILIENCE-3	Not assessed	Source is public by design
MASVS-RESILIENCE-4	Not assessed	Not expected
MASVS-PRIVACY-1	Partial	No SDKs or ad identifiers; keyboard signals wait for consent since fe379aa; direct launch requests remain (RVPN-01-406)
MASVS-PRIVACY-2	Partial	Reinstall-proof `cid`, per-install stable carrier choices (RVPN-01-410)
MASVS-PRIVACY-3	Partial	Privacy policy claims collection the code does not perform; rules upload undisclosed (RVPN-01-409)
MASVS-PRIVACY-4	Partial	Relay withdrawal effective since 7a0e32f; “Help Improve” still controls nothing (RVPN-01-405)

Appendix D: Open Questions for the Maintainers

1. Which signed configuration does the cashier serve to iOS today? In particular, does it enable `protection` or `messaging` (RVPN-01-409), the blocklist (RVPN-01-412) and MUX2; which `upgrade_hosts` pool does it carry (RVPN-01-410); and is `wgtcp` in any regional `transport_order` (RVPN-01-407, RVPN-01-413, RVPN-01-414)?
2. Which host does `BRAND_CASHIER_BASE` name in production, and is it fronted or shared with unrelated services (RVPN-01-406)?
3. The development team describes fail-open operation as a product decision (RVPN-01-402). Which side effects of `includeAllNetworks` led to it, and would a user-selectable kill switch built on `includeAllNetworks` and on-demand rules be acceptable?
4. How do the exits answer a replayed upgrade request that is not followed by a Noise handshake, and do they refuse private, loopback and link-local destinations sent through the tunnel? These are questions for WP1 and WP2 (RVPN-01-410).
5. Is one Noise static key shared by all POPs, and what is the rotation and revocation plan (RVPN-01-408)?
6. The cores shipped at bd9bbc6 were replaced by reproducible builds from rostamvpn-android 621ba38 (RVPN-01-415). Did any released build contain the earlier RedSocketCore, whose crate object contains `socks5::Upstream::active_label`, and from which source was it built?
7. Which hev-socks5-tunnel commit and lwIP revision does `libhev-socks5-tunnel.a` contain (RVPN-01-415)?
8. The project sets `ITSAppUsesNonExemptEncryption = NO` while implementing Noise and WireGuard. Has export-compliance counsel confirmed this exemption?
9. Are `registerForRemoteNotifications` (no `aps-environment` entitlement), the `BGTaskScheduler` registration (no permitted identifiers) and the `config-updated` provider message, which the extension ignores, intended to be dead code? Should a running tunnel reload an updated configuration?
10. The provider logs “using built-in endpoints” when no configuration is stored, but the core refuses to start without one. Is a bundled boot configuration intended for this brand, given that `BRAND_BUNDLED_CONFIG = NO`?
11. Will the relay remain in the iOS build? If so, what are the planned consent flow and the gateway-side authentication of the `RMUX` handshake (RVPN-01-405)?
12. Is the upload of users’ allow and deny lists to the exit described anywhere users can read it (RVPN-01-409)?
13. The post-connect check still contacts an echo service, although its answer is no longer logged (RVPN-01-403). Is that service operator-run, does it retain the addresses it echoes, and is the check still needed?

14. Which export will the WGTCP core provide for the handshake age, so that readiness and the health check can follow the first completed WireGuard handshake (RVPN-01-413)?
15. Will the WireGuard private key move into a keychain item that the extension can read, with the App Group as the keychain access group, and will the identifier and the decrypted configuration move out of the backed-up shared defaults (RVPN-01-404)?
16. When will the Proteus transport reach the iOS client, and until when will the HTTP carrier be retained for legacy clients (RVPN-01-410)?